

Section 1 — Core Fundamentals (10 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 01 Top 50

The 10 questions panels open with — to filter the "I write Python every day" candidate from the "I know what Python actually does" candidate. Indian Service MNCs (TCS, Infosys, Wipro, Accenture, Cognizant) hit these in screening rounds. Product cos (Razorpay, Swiggy, Zerodha, Cred, Flipkart, PhonePe) hit them in round 1 as warm-ups before going harder. GCC panels (Walmart Labs, JP Morgan, Goldman, Target) use them to set the tone — if you stumble here, the round is already over.

Each Q ships in the same Step-by-Step Answer format used across this file — Tags, the question verbatim, the 30-second answer, numbered Step-by-step thought process, edge cases, what NOT to say, and the common follow-up. Code blocks compile on Python 3.12+.

Note on overlap: Some Qs here have intentional overlap with later File 04 chapters — File 01 is for quick-scan, File 04 is for deep prep.

Q1 · `==` vs `is` (and `is None`)

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

What's the difference between `==` and `is` in Python? When should you actually use `is`?

The 30-second answer: `==` calls `__eq__` and asks "are these values equal?"; `is` checks object identity — "are these the exact same object in memory?" The only place you should use `is` in production is against singletons — `None`, `True`, `False` — because they're guaranteed unique. Everywhere else, use `==`.

Step-by-step thought process: 1. "Lead with the semantics. `==` is value-equality and delegates to the type's `__eq__`; `is` is the same as comparing `id(a) == id(b)`." 2. "State the cardinal rule out loud. `if x is None` is the right idiom for None-checks. Linters — ruff, pylint, flake8 — will flag `if x == None` because `__eq__` could be overridden to lie." 3. "Surface the small-int cache trap before they ask. CPython caches small integers in the range `-5` to `256`, so `a = 256; b = 256; a is b` is `True` — but `a = 257; b = 257; a is b` is `False`. That's a CPython implementation detail, not a language guarantee." 4. "Mention the string-interning trap.

Short literal strings often get interned (`"hello" is "hello"` → `True`), but dynamically-built strings don't (`"hel" + "lo" is "hello"` → `False` in many cases). Never rely on it."

Edge cases to mention: - `is` against an integer literal works inside the small-int cache and breaks outside — never use `is` for numeric comparisons. - A custom `__eq__` that returns `True` does not make `is` return `True` — identity is independent of equality. - `nan == nan` is `False` (IEEE-754); `nan is nan` is `True` when it's the same object. Both behaviours have bitten production code.

What NOT to say: - "`is` and `==` are the same for primitives" — wrong; they're the same only by coincidence inside the small-int cache. - "Use `is` because it's faster" — pointless micro-optimisation that introduces correctness bugs outside the cache range. - "`is None` is just a style preference" — it's a correctness fix; a lying `__eq__` can make `x == None` return `True` for non-None values.

Common follow-up: "Why does `a = 256; b = 256; a is b` return `True` but `a = 257; b = 257; a is b` return `False` ?" — Because CPython pre-allocates and caches integer objects in the range `-5` to `256` at interpreter startup, so two `256` s point to the same cached object. Outside that range, each literal allocates a fresh object.

```
a = 256
b = 256
print(a is b)          # True – small-int cache
a = 257
b = 257
print(a is b)          # False – outside the -5..256 range

x = None
print(x is None)       # True – correct None check
print(x == None)       # True but linter-banned
```

Stand-out line: "`is` is for singletons only — `None` , `True` , `False` . Everything else uses `==` , because identity is a memory artefact, not a contract."

Q2 · Mutability — Immutable vs Mutable Types

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Swiggy, Zerodha · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

Which types in Python are mutable and which are immutable, and why does the distinction matter day-to-day?

The 30-second answer: Immutable types (`int` , `float` , `str` , `tuple` , `frozenset` , `bytes` , `None`) cannot change after creation — any "modification" allocates a new object. Mutable types

(`list`, `dict`, `set`, `bytearray`, and most user classes) can be modified in place. This distinction drives default-argument bugs, shared-reference aliasing, and what's legal as a dict key.

Step-by-step thought process: 1. "Anchor the definition first. Mutability is about whether the object's internal state can change after `__init__`, not about whether you can rebind the name." 2. "Walk through the rebind-vs-mutate distinction. `x = 5; x = 6` rebinds `x` to a new int object — that's not mutation. `lst = [1]; lst.append(2)` is mutation — the list object's state changed; any other name pointing to that list sees the change too." 3. "Connect it to the dict-key rule. Keys must be hashable, and Python hashable-by-default means immutable. `dict[(1, 2)] = "ok"` works; `dict[[1, 2]] = "broken"` raises `TypeError: unhashable type: 'list'`." 4. "Tie it to the default-arg trap (Q3). Default values are evaluated once at `def` time, so a mutable default is shared across every call — the textbook bug that bites in production handlers."

Edge cases to mention: - A `tuple` containing a list (`(1, [2, 3])`) is technically immutable at the tuple level but still unhashable — Python checks recursively. - `str` is immutable, so `s += "x"` in a loop builds a new string every iteration — $O(n^2)$. Use `"".join(parts)` instead. - `frozenset` exists specifically because `set` isn't hashable and you sometimes need a set-as-dict-key.

What NOT to say: - "`int` is mutable because I can do `x += 1`" — no; `x += 1` rebinds `x` to a new int object, the original is untouched. - "Tuples are always hashable" — wrong; only when every contained element is hashable. - "Mutating a `str` is just slow" — wrong; you can't mutate a `str` at all. Every "edit" allocates a new object.

Common follow-up: "If `tuple` is immutable, why can `(1, [2, 3])` still see its inner list change?" — Because the tuple stores references, and the references themselves don't change — but the objects they point to can if they're mutable. The tuple is immutable; the contents need not be.

```
# Immutable: int rebinds, doesn't mutate
x = 5
y = x
x += 1
print(x, y)           # 6 5 — y unchanged

# Mutable: list aliases share state
a = [1, 2]
b = a
b.append(3)
print(a, b)           # [1, 2, 3] [1, 2, 3] — same object

# Hashability follows immutability
d = {(1, 2): "tuple-key works"}
# d[[1, 2]] = "fails"   # TypeError: unhashable type: 'list'
```

Stand-out line: "Mutability isn't a property of the variable — it's a property of the object the variable points to. Rebinding is free; mutation is shared."

Q3 · Mutable Default Arguments — The Classic Trap

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Razorpay, Cred · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

What's wrong with `def f(x=[]):` ? Walk me through the bug and the fix.

The 30-second answer: A function's default arguments are evaluated **once**, at function-definition time, and stored on the function object. If the default is a mutable object (`list` , `dict` , `set`), every call that doesn't override it shares the **same** mutable instance — so prior calls leak into later ones. The fix is to default to `None` and create the mutable inside the function body.

Step-by-step thought process: 1. "State the mechanism precisely. `def f(x=[])` doesn't mean 'x defaults to a fresh `[]` on each call.' It means 'x defaults to the list object that was created when this `def` statement ran.' That list lives on `f.__defaults__` ." 2. "Show the failure path. Call `f()` once, mutate `x` , call `f()` again — the mutation is still there. This is the single most-asked Python gotcha in Indian screening rounds." 3. "Give the fix in one line. `def f(x=None): x = [] if x is None else x` . Or with types: `def f(x: list | None = None): x = x or []` ." 4. "Explain why it bites in production. It looks correct, passes single-call testing, and only surfaces when a second request hits the same handler and inherits the previous request's leftover state."

Edge cases to mention: - Same trap with `dict` (`def f(d={}):`) and `set` (`def f(s=set()):`). Any mutable default is wrong. - Immutable defaults are safe — `def f(x=0)` , `def f(s="")` , `def f(t=())` all work as expected because you can't mutate them. - Linters (ruff `B006` , pylint `dangerous-default-value`) flag this automatically — turn that rule on in CI.

What NOT to say: - "Just don't mutate the argument" — fragile contract; the next maintainer will mutate it. Fix the default instead. - "Use `copy.copy(default)` inside the function" — works but heavier than `None` -and-create. The `None` sentinel is the idiomatic fix. - "The default is evaluated on every call" — wrong; defaults are evaluated once at `def` time and that's the whole bug.

Common follow-up: "How would you handle a default that's an expensive immutable, like a precomputed lookup table?" — Default it to `None` and lazily compute inside the function, caching on a module-level variable. Or use `functools.lru_cache` on a builder function.

```
def add_item(item, basket=[]):          # BUG: shared default
    basket.append(item)
    return basket

print(add_item("a"))                    # ['a']
print(add_item("b"))                    # ['a', 'b'] — leaked!

def add_item_fixed(item, basket=None):
    basket = [] if basket is None else basket
    basket.append(item)
    return basket

print(add_item_fixed("a"))              # ['a']
print(add_item_fixed("b"))              # ['b'] — fresh
```

Stand-out line: "The default is evaluated at `def` time, not call time — that one sentence kills the bug and proves you've read the data model."

Q4 · List Comprehension vs Generator Expression

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Swiggy, Postman · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

What's the difference between `[x*x for x in xs]` and `(x*x for x in xs)` ? When do you pick each?

The 30-second answer: A list comprehension builds the entire list in memory eagerly; a generator expression returns a lazy iterator that yields values one at a time. Use the list when you need the whole result, can iterate it more than once, or need indexing. Use the generator when the result is huge, you only iterate once, or you're piping into `sum` / `any` / `max` /etc.

Step-by-step thought process: 1. "Open with the syntactic difference — it's just the brackets. `[x*x for x in range(10)]` is a list; `(x*x for x in range(10))` is a generator." 2. "Move to the memory difference, which is the whole point. `sum(x*x for x in range(10**7))` uses constant memory; the list version allocates 10 million ints upfront." 3. "Explain laziness. A generator runs the `for` body lazily and stops after the consumer stops. Side effects don't happen until you pull values." 4. "End with the one-shot warning. A generator is exhausted after a single iteration — iterating twice gives empty results. A list can be walked any number of times."

Edge cases to mention: - `sum([x for x in iterable])` is wasteful — the list is built only to be summed. Drop the brackets: `sum(x for x in iterable)` . - `len(gen)` doesn't work —

generators have no length until consumed. Materialise to a list first if you need length. - Multiple-pass algorithms (computing `max` and `min` of the same source) need a list, or two passes over the source, or `itertools.tee`.

What NOT to say: - "List comprehensions are always slower than for-loops" — wrong; they're typically faster and the bytecode is tighter. - "Generators save you CPU time" — they save memory; per-element work is the same. - "Just use a list; you can always re-iterate" — costs memory you don't need to spend for one-shot pipelines like `sum`, `any`, `max`.

Common follow-up: "If I wrap `sum(...)` around a generator expression, am I losing the readability of the list comprehension?" — No; `sum(x*x for x in xs)` is the canonical Python form and reads cleaner than the bracketed version. The brackets only earn their keep when you'll iterate more than once.

```
# List – eager, materialised, re-iterable
nums = [x * x for x in range(5)]
print(nums)                # [0, 1, 4, 9, 16]
print(sum(nums), sum(nums)) # 30 30 – re-iterable

# Generator – lazy, one-shot
gen = (x * x for x in range(5))
print(sum(gen))             # 30
print(sum(gen))             # 0 – already exhausted

# The memory win
total = sum(x * x for x in range(10**7)) # constant memory
```

Stand-out line: "List comprehensions materialise; generators stream. The choice is memory vs reusability — pick deliberately, not by reflex."

Q5 · Dict / Set Comprehensions and the Walrus Inside Them

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Postman, Walmart Labs · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

Show me a dict comprehension and a set comprehension. What does the walrus operator buy you inside one?

The 30-second answer: Same shape as a list comprehension, different brackets.

`{k: v for k, v in items}` builds a dict; `{x for x in items}` builds a set; `{}` alone is an empty dict (not a set — empty set is `set()`). The walrus operator `:=` inside a comprehension lets you compute a value once and reuse it in both the filter and the result.

Step-by-step thought process: 1. "List the four comprehension forms — list `[]`, set `{x}`, dict `{k: v}`, generator `()`. They all evaluate left-to-right — outer-loop first, then inner loops, then condition, then expression." 2. "Show the dict-comp use case. Inverting a mapping, transforming values, or projecting a subset: `{k: v.upper() for k, v in d.items() if v}`." 3. "Show the set-comp use case — auto-dedupe and transform. `{c.lower() for c in "Hello"}` returns `{'h', 'e', 'l', 'o'}`." 4. "Bring in the walrus. PEP 572, Python 3.8+. It's the only way to compute-once-use-twice cleanly inside a comprehension — without it you'd call the expensive function twice or pre-build a list."

Edge cases to mention: - `{}` is an empty dict, not an empty set — Python had dict literals before set literals, so dict won the syntax. - Dict comprehensions silently dedupe on the key:

`{k: v for k, v in [(1, "a"), (1, "b")]}` returns `{1: "b"}` — last write wins. - The walrus binds the name in the enclosing scope of the comprehension (the function or module), not inside the comp — that's specified in PEP 572 and trips people in nested comps.

What NOT to say: - "`{}` is an empty set" — wrong; empty set is `set()`. Saying this in a senior round is a credibility hit. - "Walrus inside a comp is just syntactic sugar for a normal assignment" — it's specifically an expression-level assignment that legalises capture-and-test in one shot; a normal `=` is a statement and can't be embedded. - "Dict comprehensions preserve all keys" — they don't; duplicates silently overwrite.

Common follow-up: "Why does the walrus matter in a comprehension specifically?" — Because comprehensions don't permit statements; the only way to filter on an expensive computation and also use its value in the result is to compute it twice — unless you bind it once via `(y := expensive(x))`.

```
# Dict comprehension — invert a mapping
prices = {"tea": 20, "coffee": 80}
by_price = {v: k for k, v in prices.items()}
print(by_price)                    # {20: 'tea', 80: 'coffee'}

# Set comprehension — dedupe + transform
seen = {c.lower() for c in "TCS Infosys"}
print(sorted(seen))               # [' ', 'c', 'f', 'i', ...]

# Walrus inside a comprehension — avoid double work
import re
lines = ["error: 500", "ok: 200", "error: 503"]
errors = [m.group(0) for line in lines if (m := re.match(r"error: \d+", line))]
print(errors)                    # ['error: 500', 'error: 503']
```

Stand-out line: "The walrus inside a comprehension exists for one reason: filter on an expensive expression, then return it without recomputing. That's the question they're testing."

Q6 · `*args` / `**kwargs` and the `/`, `*` Markers

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Cred, Goldman, Walmart Labs · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

What do `*args` and `**kwargs` do? What about a bare `/` or bare `*` in the signature?

The 30-second answer: `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. At call sites, `*` and `**` unpack them back out. In the signature, a bare `/` marks everything before it as positional-only; a bare `*` marks everything after it as keyword-only. Together they give you precise control over how callers pass arguments.

Step-by-step thought process: 1. "Separate the two contexts. In the function definition, `def f(*args, **kwargs)` collects extras. At the call site, `f(*lst, **d)` unpacks. Same symbol, opposite direction." 2. "Bring in positional-only `/`. PEP 570, Python 3.8+. Built-ins like `len(obj, /)` have used it forever — useful when the parameter name is meaningless or you want to rename it freely without breaking callers." 3. "Bring in keyword-only `*`. The safer signature pattern — `def transfer(amount, *, currency='INR')` forces callers to write `currency='USD'` explicitly. Removes the 'what does the third positional argument mean again?' reading bug." 4. "State the full ordering. `def f(pos_only, /, pos_or_kw, *args, kw_only, **kwargs)`. Senior interviewers will ask you to construct this verbatim — memorise it."

Edge cases to mention: - `*args` must come before `**kwargs` in the signature — Python enforces it syntactically. - A bare `*` in the signature without a name (`def f(a, *, b)`) is a marker, not a collector. `def f(a, *rest, b)` is the collector form. - Unpacking a dict into `**kwargs` raises `TypeError` if a key collides with an existing positional argument — Python won't let you pass `transfer(500, "ACC1", **{"account": "ACC2"})`.

What NOT to say: - "`*args` and `**kwargs` are the same thing with different syntax" — wrong; one collects positionals, the other collects keywords. Mixing them up costs a senior round instantly. - "I always use `*args, **kwargs` for flexibility" — anti-pattern; it hides the real signature from the reader and the type checker. - "Bare `*` is just a stylistic separator" — it's a hard contract — anything after it is keyword-only or Python raises `TypeError` at the call site.

Common follow-up: "When would you actually use positional-only `/`?" — When the parameter name is meaningless to the caller (e.g. `len(obj, /)`), when you want to rename the parameter freely without breaking callers, or when you want to allow the same name as a `**kwargs` key without collision.


```
def transfer(amount, /, account, *, currency="INR", **meta):
    return f"{amount} {currency} -> {account}, meta={meta}"

# Positional-only: amount must be positional
print(transfer(500, "ACC1"))
# transfer(amount=500, account="ACC1") # TypeError: amount is pos-only

# Keyword-only: currency must be keyword
# transfer(500, "ACC1", "USD") # TypeError: too many pos args
print(transfer(500, "ACC1", currency="USD", note="refund"))

# Unpacking at call site
args = (500, "ACC1")
kwargs = {"currency": "USD", "ref": "TXN123"}
print(transfer(*args, **kwargs))
```

Stand-out line: `/` and `*` aren't optional cosmetics — they're the signature contract. Bare `*` forces keyword-only so callers can't pass `True` and leave the next reader guessing what it meant."

Q7 · Lambda — When to Use, When Not, Closure Semantics

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Postman, Flipkart · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

When is a lambda the right tool, and when is it the wrong one? Walk me through the late-binding closure bug.

The 30-second answer: A lambda is a single-expression anonymous function. Use it inline as a one-shot `key=` for `sorted` / `min` / `max`, as a small callback, or for `functools.partial`-style binding. Don't use it when you'd have to name it — if you write `f = lambda x: ...`, just write `def f(x): ...` instead; the named `def` gives a real `__name__` for tracebacks and supports docstrings.

Step-by-step thought process: 1. "Open with the constraint. Lambda is restricted by design — one expression, no statements (no `return`, no `if/elif/else`, no assignments — though `:=` walrus works since 3.8). Anything that needs control flow needs `def`." 2. "Give the canonical use. Inline sort keys: `sorted(orders, key=lambda o: o.amount)`. Clean, local, throwaway." 3. "Spring the late-binding trap. `funcs = [lambda: i for i in range(3)]` then `[f() for f in funcs]` returns `[2, 2, 2]`, not `[0, 1, 2]`, because each lambda closes over the variable `i`, not its value at the time the lambda was created." 4. "Give the one-line fix. Default-argument binding: `funcs = [lambda i=i: i for i in range(3)]` captures the current value. Or use `functools.partial`."

Edge cases to mention: - PEP 8 explicitly says don't assign lambdas to names — use `def`. Linters (ruff `E731`) flag it. - Closures close over variables, not values. The variable is looked up when the lambda runs, not when it's defined. - A lambda inside a comprehension that uses the comp variable hits the same late-binding trap as the loop version — same fix.

What NOT to say: - "Lambdas are faster than `def`" — they're the same bytecode under the hood; pick by readability, not speed. - "Closures capture values at definition time" — wrong; they capture variables, and the variable is looked up when the closure runs. - "The bug is a Python quirk" — it's a documented language semantic that mirrors how every closure-capturing language behaves; calling it a quirk reads as undergrad.

Common follow-up: "Why does `funcs = [lambda i=i: i for i in range(3)]` fix the late-binding bug?" — Because default arguments are evaluated at function-definition time, so the `i=i` binds the current value of `i` to a new parameter scoped to the lambda. The body's `i` then refers to the parameter, not the outer loop variable.

```
# Good lambda — inline sort key
orders = [{"id": 1, "amt": 500}, {"id": 2, "amt": 200}]
print(sorted(orders, key=lambda o: o["amt"]))

# Bad lambda — assigned to a name; use def
add = lambda a, b: a + b          # bad: __name__ is '<lambda>'
def add_v2(a, b): return a + b    # good: __name__ is 'add_v2'

# Late-binding closure trap
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])      # [2, 2, 2] — all see final i

# Fix with default-argument binding
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])      # [0, 1, 2]
```

Stand-out line: "If I'm naming the lambda I've already lost — `def` is shorter to type and gives me a real name in tracebacks. Lambdas are for inline, not for hiding `def` from yourself."

Q8 · Truthiness — What's Falsy in Python

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Razorpay · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

Which values are falsy in Python? What protocol does `if x:` actually invoke?

The 30-second answer: Python has exactly seven categories of falsy values: `False`, `None`, numeric zeros (`0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0)`), empty sequences (`""`, `[]`, `()`, `b""`, `range(0)`), empty mappings (`{}`), and empty sets (`set()`, `frozenset()`). Everything else is truthy. `if x:` calls `x.__bool__()` or, failing that, `x.__len__() != 0`.

Step-by-step thought process: 1. "Open by listing the categories — they want to hear you enumerate, not handwave. In interviews you'll be asked to predict the truthiness of `if [0]:`, `if {None}:`, `if " ":`, `if 0.0:`. Containers are falsy when empty regardless of what falsy items they would contain if non-empty." 2. "Walk the protocol. `bool(x)` tries `__bool__` first, then falls back to `__len__`. If neither is defined, the object is truthy. That's why user classes are truthy by default." 3. "Surface the real-world bug. `if user.name:` treats an empty string as missing. If the field is genuinely optional, check `if user.name is not None` — empty-string-as-falsy lumps two different states together." 4. "Give the idiomatic default-fallback. `if user.name or "Anon":` — `or` returns the first truthy operand, not a boolean."

Edge cases to mention: - `0.0 == False` is `True` and `0 == False` is `True` — Python's `bool` is a subclass of `int`. So `True + True == 2` works. - `numpy` arrays raise `ValueError` on `if arr:` for arrays with more than one element — you need `arr.any()` or `arr.all()` explicitly. - `pandas` series and dataframes do the same — calling `if df:` raises. Always be explicit with `df.empty`, `len(df)`, etc.

What NOT to say: - "Empty containers are still truthy because they exist" — wrong; emptiness is the entire criterion for built-in containers. - "`bool(x)` is the same as `x != 0`" — only true for numbers; for containers it asks "is this empty?", for user classes it asks `__bool__` or `__len__`. - "Just use `if x is True` to be safe" — anti-pattern; that breaks for any value that isn't literally `True`, including `1` and `[1]`.

Common follow-up: "Why does `if user.name:` lump empty string and missing together — and how would you separate them?" — Because `""` and `None` are both falsy, the test can't distinguish them. Replace it with `if user.name is not None` (or `if user.name not in (None, "")` if you want both blank and missing to fall through).

```
# All seven falsy categories
for v in [False, None, 0, 0.0, "", [], (), {}, set(), b"", range(0)]:
    print(repr(v), bool(v))          # all False

# Truthy surprises
print(bool([0]))                    # True — non-empty container
print(bool([False]))                # True — non-empty
print(bool(" "))                     # True — whitespace string

# Custom __bool__
class Cart:
    def __init__(self, items): self.items = items
    def __bool__(self): return len(self.items) > 0

print(bool(Cart([])), bool(Cart(["x"]))) # False True
```

Stand-out line: "Truthiness lumps `None`, empty, and zero into one bucket — that's a feature for terse code and a bug for state checks. When the difference matters, write `is None` or `len(x) == 0` explicitly."

Q9 · Identity vs Equality — `id()`, `__eq__`, `__hash__`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan, Walmart Labs · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

If I override `__eq__` on a class, what happens to `__hash__` — and why does Python do that?

The 30-second answer: Identity (`is`, `id()`) asks "same object?"; equality (`==`, `__eq__`) asks "same value?" If you override `__eq__` on a class you must also override `__hash__` — Python sets `__hash__` to `None` automatically when you define `__eq__`, making your class unhashable until you fix it. Two objects that compare equal must have the same hash, or dicts and sets break.

Step-by-step thought process: 1. "State the defaults. Object identity is the default for `__eq__` and `__hash__` — distinct instances are unequal and have distinct hashes derived from `id()` ." 2. "Say the hash contract out loud. `a == b` implies `hash(a) == hash(b)` . The reverse is not required — collisions are fine. Violate the forward direction and a dict lookup that should find the key won't." 3. "Explain why Python auto-nulls `__hash__` . If you define `__eq__` based on a field and that field is mutable, equality can change after the object is in a set — corrupting the set's bucket assumptions. So Python forces you to opt in." 4. "Give the modern shortcut. `dataclass(frozen=True)` generates both `__eq__` and `__hash__` correctly for you — the recommended path for value objects."

Edge cases to mention: - `is` for sentinel checks (`is None`, `is _MISSING`) is correct and idiomatic. `is` for arbitrary value comparison (`a is 5`) is wrong. - Overriding `__eq__` without `__hash__` is the most common cause of "my Django model lost its hash" — Django and SQLAlchemy explicitly handle this; rolling your own class doesn't get the same protection. - `id()` returns the same value for two non-overlapping object lifetimes — never store `id()` for "remember this object."

What NOT to say: - "I'll just return `id(self)` from `__hash__` " — breaks the contract — `a == b` but `hash(a) != hash(b)`, so sets and dicts misbehave. - "Defining `__eq__` alone is fine" — only if you never put the instance in a set or dict key, and the failure mode (unhashable) is silent. - "`__hash__` only matters for performance" — wrong; it's a correctness contract; violating it makes set membership unpredictable.

Common follow-up: "What does Q29's deeper `__eq__` / `__hash__` contract question add on top of this one?" — Q29 walks through `NotImplemented` vs `False` from `__eq__`, why mutable types

are unhashable, and how `@dataclass(frozen=True)` writes both methods correctly. Q9 is the introduction; Q29 is the contract exam.

```
from dataclasses import dataclass

class Account:
    def __init__(self, num): self.num = num
    def __eq__(self, other):
        return isinstance(other, Account) and self.num == other.num
    # __hash__ now None – class is unhashable!

a1, a2 = Account("X"), Account("X")
print(a1 == a2)                # True
# {a1, a2}                     # TypeError: unhashable type: 'Account'

# Frozen dataclass – __eq__ and __hash__ both generated
@dataclass(frozen=True)
class AccountV2:
    num: str

print({AccountV2("X"), AccountV2("X")}) # {AccountV2(num='X')} – deduped
```

Stand-out line: "If I'm writing `__eq__`, I'm thinking about `__hash__` in the same breath — they're a pair. Define one, define both, or use `frozen=True` and let `dataclass` write both."

Q10 · The Walrus Operator `:=`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Cred · Topic: Core Fundamentals

The question (interviewer's verbatim phrasing):

What is `:=`, what version added it, and when does it actually pay for itself over a normal assignment?

The 30-second answer: `:=` (PEP 572, Python 3.8+) is the assignment expression — it lets you assign and use a value in the same expression. Its real job is removing duplicated work in `while` loops, comprehensions, and `if` conditions where you'd otherwise call a function twice or pre-bind a variable on a separate line.

Step-by-step thought process: 1. "Open with the classic non-walrus pattern. `data = f.read(); while data: process(data); data = f.read()` — `f.read()` appears twice. Walrus rewrites it to `while data := f.read(): process(data)`." 2. "Bring the `if` use case. `if (m := re.match(pattern, s)): use(m.group(1))` — match once, use the result." 3. "Mention the comprehension use case. Filter and transform on the same computed value without recomputing — see

Q5's code example." 4. "State when not to use it. Simple assignments stay as `=` statements. The walrus is for the cases where the language otherwise forces you into duplication or an extra line that obscures intent."

Edge cases to mention: - Walrus requires parentheses in many contexts: `if (n := len(data)) > 100:` — without the parens, the precedence is wrong. - It binds in the enclosing scope, not the comprehension's local scope — so `[y for x in r if (y := f(x))]` leaks `y` into the surrounding function. That's intentional, but surprising. - Don't use it in a list comprehension just because you can — `[(x, x*x) for x in r]` is clearer than `[(x, y := x*x) for x in r]` when there's no filtering.

What NOT to say: - "Walrus is just shorter `=`" — wrong; `=` is a statement and can't appear inside expressions, which is exactly the problem walrus solves. - "Walrus is Pythonic everywhere" — no; using it where a normal assignment fits adds noise without saving a line. - "Walrus doesn't work inside comprehensions" — it does, and that's one of its highest-value use cases.

Common follow-up: "Show me a walrus in a `while` loop that reads a file in chunks." — `while chunk := f.read(1024): process(chunk)`. The loop reads, binds, and tests in one expression; when `read` returns an empty bytes object the loop exits cleanly.

```
import re

# Walrus in while — single read, no duplication
def read_chunks(path, size=1024):
    with open(path) as f:
        while chunk := f.read(size):
            yield chunk

# Walrus in if — capture and check
log = "error 503 — service unavailable"
if (m := re.search(r"error (\d+)", log)):
    print(m.group(1))          # 503

# Walrus in comprehension — compute once, filter + use
values = [1, 2, 3, 4, 5]
squared = [y for x in values if (y := x * x) > 5]
print(squared)                # [9, 16, 25]
```

Stand-out line: "The walrus exists for one thing — eliminating the compute-twice-or-pre-bind dance. If the code doesn't have that smell, the walrus is just noise."

End of Section 1 (10 Qs). Section 2 covers decorators, generators, and iterators next.

Section 2 — Decorators, Generators, Iterators

Ten questions that decide whether you sound like someone who *writes* Python or someone who *understands* it. Decorators trip up confident devs because the syntax hides the call order; generators trip them up because lazy evaluation behaves differently from lists in ways nobody bothers to test. The interviewer here is sniffing for "this person reads CPython source on the weekend" or "this person learned Python by copy-pasting from Stack Overflow." Each question below gives you the 30-second answer first, then how to walk through it out loud, then the gotcha they're hoping you miss.

Q11 · What Is a Decorator

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Decorators

The question (interviewer's verbatim phrasing):

What is a decorator in Python? Show me what `@my_decorator` actually does under the hood.

The 30-second answer: A decorator is a callable that takes a function and returns a (usually new) function. The `@decorator` syntax above a `def` is pure sugar for `f = decorator(f)` — the function is defined, immediately passed to the decorator, and the original name is rebound to whatever the decorator returns.

Step-by-step thought process: 1. Open with the definition that matters: "function-that-takes-function-returns-function." That one phrase tells the interviewer you understand callables-as-values. 2. Show the desugaring out loud. `@log` above `def greet(): ...` is exactly `greet = log(greet)` — Python evaluates the `def`, hands the function object to `log`, and rebinds the name. The `@` is not magic; it's syntax. 3. State what a typical decorator returns: a *wrapper* function that calls the original inside it, with extra behaviour around the call. Logging, timing, auth, retry — all the same shape. 4. Anticipate the follow-up about side effects. The decorator runs *once* at definition time. The wrapper runs every time the decorated function is called. Don't confuse those two timings.

Edge cases to mention: - The decorator doesn't have to return a wrapper — it can return the original function unchanged (a registration-style decorator that just records the function in a list). - A decorator can return a completely different object — a class instance, a partial, anything callable. - If the decorator returns `None` (forgetting the `return`), the decorated name becomes `None` and calling it raises `TypeError: 'NoneType' object is not callable`. Easy mistake during refactors.

What NOT to say: - "A decorator adds functionality" — too vague, sounds like a tutorial. State the call signature explicitly. - "It modifies the function" — it doesn't modify; it replaces the name binding. Subtle but the interviewer notices.

Common follow-up: "Write me a `@timing` decorator that prints how long the function took."


```
import time

def timing(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

@timing # equivalent to: slow_thing = timing(slow_thing)
def slow_thing(n: int) -> int:
    return sum(range(n))

slow_thing(1_000_000)
```

Stand-out line: "The `@` is just sugar for `f = decorator(f)` — once you see it that way, every decorator question becomes a function-composition question."

Q12 · Decorator Stacking Order

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs · Topic: Decorators · Lander sample card 01

The question (interviewer's verbatim phrasing):

If I write `@a` above `@b` above `def f(): ...`, which decorator runs first when I call `f()` ?

The 30-second answer: At definition time, the innermost decorator wraps first: `@a @b def f` evaluates as `f = a(b(f))`. At call time, the *outermost* wrapper runs first — `a`'s wrapper executes, calls into `b`'s wrapper, which finally calls the original `f`. So definition is bottom-up; execution is top-down.

Step-by-step thought process: 1. Separate the two phases — that's the whole trick. Definition time and call time behave like opposite ends of a stack. 2. Walk through the desugaring: Python sees `def f` first (innermost line), then applies `@b` to get `b(f)`, then applies `@a` to get `a(b(f))`. The name `f` is now bound to `a`'s wrapper. 3. When you call `f()`, you're calling `a`'s wrapper. Inside, it calls `b`'s wrapper. Inside *that*, it calls the original function. The print/log lines from `a` appear before `b`'s. 4. The interview mnemonic: "*decorators apply bottom-up, execute top-down.*" Say it exactly like that.

Edge cases to mention: - If `a` returns *without* calling the wrapped function (e.g. an auth decorator that rejects the call), `b`'s wrapper and the original function never run. Order matters for short-circuit behaviour. - A decorator that mutates arguments before calling through (e.g. `@validate`) affects what

b and the original see — put validation outermost. - Exception flow runs in reverse — exceptions raised by the original function bubble up through **b** 's wrapper first, then **a** 's. Useful for retry / logging decorators.

What NOT to say: - "They run in the order they're written" — ambiguous; the interviewer is testing whether you can separate define-time from call-time. - "The order doesn't matter" — wrong; auth-before-cache vs cache-before-auth changes security.

Common follow-up: "You have `@cache` and `@requires_auth` . Which one goes on top, and why?" — Auth on top. If auth fails, you never want to touch the cache (or worse, return cached data to an unauthorised caller).

```
def outer(func):
    def wrapper(*a, **k):
        print("outer in")
        result = func(*a, **k)
        print("outer out")
        return result
    return wrapper

def inner(func):
    def wrapper(*a, **k):
        print("inner in")
        result = func(*a, **k)
        print("inner out")
        return result
    return wrapper

@outer # applied second -> outer(inner(hello))
@inner # applied first
def hello() -> None:
    print("hello")

hello()
# outer in
# inner in
# hello
# inner out
# outer out
```

Stand-out line: "Decorators apply bottom-up at definition time and execute top-down at call time — same stack, read from opposite ends."

Q13 · `functools.wraps` — Why You Need It

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Zoho, Freshworks · Topic: Decorators

The question (interviewer's verbatim phrasing):

Why does every decorator example use `@functools.wraps(func)` on the inner wrapper?
What breaks if you skip it?

The 30-second answer: A wrapper function has its own `__name__`, `__doc__`, `__module__`, `__qualname__`, and `__wrapped__` — so the decorated function "loses its identity" and reports as `wrapper`. `functools.wraps` copies those metadata attributes from the original onto the wrapper, so `help()`, debuggers, logging frameworks, and `inspect` keep seeing the real function.

Step-by-step thought process: 1. Lead with what breaks. `print(greet.__name__)` returns `'wrapper'` instead of `'greet'`. `help(greet)` shows the wrapper's empty docstring. Pytest fixture introspection, FastAPI's signature scanning, sphinx-autodoc — all of it goes wrong. 2. Explain the fix mechanically. `@functools.wraps(func)` is itself a decorator that copies `__module__`, `__name__`, `__qualname__`, `__annotations__`, `__doc__`, and updates `__dict__`. It also sets `__wrapped__ = func` so introspection can recover the original. 3. Call out the real-world failure mode: production frameworks like FastAPI use `inspect.signature` to build request schemas. A bare wrapper has signature `(*args, **kwargs)` — your API docs become "accepts anything" and Pydantic validation breaks. 4. Mention `functools.update_wrapper` as the function-call form — same thing without the decorator sugar, useful when you build a wrapper imperatively.

Edge cases to mention: - `wraps` copies references, not deep copies — mutating `wrapper.__dict__` mutates only the wrapper, but mutating `__annotations__` mutates the dict that was copied over (it's the same object). - Class-based decorators (Q15) need to call `functools.update_wrapper(self, func)` inside `__init__` — there's no `@wraps` shortcut for instances. - `__wrapped__` lets `inspect.signature` see through the wrapper by default — useful, but if you intentionally changed the signature, pass `inspect.signature(f, follow_wrapped=False)` to see what the wrapper actually accepts.

What NOT to say: - "It's just cosmetic" — wrong; FastAPI, pytest, sphinx, and pickle all break in non-cosmetic ways. - "You only need it for docs" — incomplete; debuggers and signature inspection rely on it too.

Common follow-up: "What does `wraps` copy, exactly?" — `__module__`, `__name__`, `__qualname__`, `__annotations__`, `__doc__`; updates `__dict__`; sets `__wrapped__`. Defined in `functools.WRAPPER_ASSIGNMENTS` and `functools.WRAPPER_UPDATES`.

```
import functools

def log_calls(func):
    @functools.wraps(func)          # without this, greet.__name__ == 'wrapper'
    def wrapper(*args, **kwargs):
        print(f"calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_calls
def greet(name: str) -> str:
    """Return a friendly greeting."""
    return f"Hello, {name}"

print(greet.__name__)    # 'greet'
print(greet.__doc__)     # 'Return a friendly greeting.'
print(greet.__wrapped__) # <function greet at 0x...> -> the original
```

Stand-out line: " `functools.wraps` isn't cosmetic — it's how FastAPI, pytest, and every signature-inspecting framework see the real function instead of your wrapper."

Q14 · Parameterized Decorators

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, JP Morgan · Topic: Decorators

The question (interviewer's verbatim phrasing):

How do you write a decorator that accepts arguments, like `@retry(times=3)` ? Walk me through the nesting.

The 30-second answer: You add one more layer. A normal decorator is one function taking a function; a parameterized decorator is a *factory* — an outer function that takes the arguments, returns a decorator, which then takes the function and returns the wrapper. Three nested functions instead of two.

Step-by-step thought process: 1. Show the desugaring first. `@retry(times=3)` above `def f` is `f = retry(times=3)(f)`. The interviewer wants to see that you read the parentheses as a call that *returns* a decorator. 2. Walk the three layers out loud. Outer `retry(times)` runs with the config args, returns the *real* decorator. Middle `decorator(func)` runs with the function being decorated, returns the wrapper. Inner `wrapper(*args, **kwargs)` runs on every call. Three scopes; closures capture `times` and `func`. 3. State the rule of thumb. If the decorator takes config arguments, you need this three-layer pattern. If it doesn't, you need the two-layer pattern. There's no way around the extra layer — Python's syntax forces it. 4. Anticipate the trick question: "What if I want it to work both with and without arguments — `@retry` and `@retry(times=3)` ?" That requires a conditional outer layer that detects

whether the first arg is a function or a config value. Mention `functools.wraps`-style libraries (or `wrapt`) handle this cleanly; doing it by hand is error-prone.

Edge cases to mention: - Always put `@functools.wraps(func)` on the innermost wrapper, not the middle layer — the middle layer never gets called; only the wrapper does. - Mutable config captured in the outer closure is shared across all decorated functions — usually a bug; either copy or document. - The "both forms" idiom is brittle; modern codebases just always require the parens (`@retry()`) for consistency.

What NOT to say: - "It's the same as a normal decorator" — wrong; the extra factory layer is mandatory. - "Just use `*args, **kwargs` everywhere" — vague; the interviewer wants to see you understand the three distinct scopes.

Common follow-up: "Write me a `@retry(times=3, on=ValueError)` decorator that retries on a specific exception."

```
import functools

def retry(times: int = 3, on: type[BaseException] = Exception):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except on as exc:
                    last_exc = exc
                    print(f"attempt {attempt} failed: {exc}")
            raise last_exc # exhausted retries
        return wrapper
    return decorator

@retry(times=3, on=ValueError)
def flaky(x: int) -> int:
    if x < 0:
        raise ValueError("negative")
    return x * 2

flaky(5) # works first try
# flaky(-1) # tries 3 times, then re-raises
```

Stand-out line: "A parameterized decorator is a decorator factory — outer takes config, middle takes the function, inner takes the call args. Three layers, three scopes, one closure chain."

Q15 · Class Decorators — Function vs Class

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Goldman, Walmart Labs · Topic: Decorators

The question (interviewer's verbatim phrasing):

When would you write a decorator as a class instead of a function? Show me one.

The 30-second answer: Use a class when the decorator needs state across calls — a hit counter, a rate limiter, a cache. The class defines `__init__` (captures the wrapped function) and `__call__` (runs on every invocation). Functions with closures *can* hold state, but a class makes the state explicit, easier to inspect, and easier to add methods to (`reset()` , `stats()`).

Step-by-step thought process: 1. State the decision rule. Stateless or one-shot side-effect → function. State that needs to be inspected / mutated / reset from outside → class. 2. Show the shape.

`__init__(self, func)` captures the wrapped function and initialises state. `__call__(self, *args, **kwargs)` is the wrapper — it's what runs when someone calls the decorated function. 3. Mention the metadata trap: `@functools.wraps` won't work as a decorator on `__call__` because the instance, not `__call__`, is what's bound. Call `functools.update_wrapper(self, func)` inside `__init__` instead. 4. Anticipate the deeper question: "Can a class be used as a *parameterized* decorator?" Yes — `__init__` takes the config args, and you add a separate `__call__` method that's the actual decorator (taking the function). Same three-layer pattern as Q14, expressed differently.

Edge cases to mention: - A class decorator applied to a *method* loses the `self` binding because the descriptor protocol doesn't see through the class instance. Workaround: implement `__get__` on the decorator class, or stick with function-based decorators for methods. - The decorator instance is shared across all decorated functions if you reuse it — usually not what you want. Re-instantiate per function. - Class-based decorators are easier to pickle if you need to serialize decorated callables — bare closures are not picklable.

What NOT to say: - "Always use classes for decorators" — no; for stateless logging/timing, a function is shorter and clearer. - "It's faster" — usually slower, actually; closures are slightly leaner than instance method dispatch.

Common follow-up: "Write a `@counted` class decorator that tracks how many times the function was called and exposes a `.count` attribute."

```
import functools

class Counted:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        return self.func(*args, **kwargs)

    def reset(self) -> None:
        self.count = 0

@Counted
def fetch(url: str) -> str:
    return f"GET {url}"

fetch("https://example.com")
fetch("https://example.com")
print(fetch.count)    # 2
fetch.reset()
print(fetch.count)    # 0
print(fetch.__name__) # 'fetch' -- thanks to update_wrapper
```

Stand-out line: "Pick a class when the decorator owns state you'd want to introspect or reset — the instance becomes the natural place to hang `.count`, `.cache`, or `.reset()`."

Q16 · Generators vs Iterators

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Generators · Lander sample card 04

The question (interviewer's verbatim phrasing):

What's the difference between an iterator and a generator? Why does the language even have both?

The 30-second answer: An iterator is any object implementing the iterator protocol — `__iter__` returning self and `__next__` returning the next value (or raising `StopIteration`). A generator is a function containing `yield` that, when called, returns a generator object — which *is* an iterator, built for you by the interpreter. Generators are how you skip writing the protocol by hand.

Step-by-step thought process: 1. State the protocol first. The iterator protocol is two methods:

`__iter__` (returns self) and `__next__` (returns the next value or raises `StopIteration` when done). Anything that implements both is an iterator. 2. Show the hand-rolled iterator class — five lines of `__init__`, `__iter__`, `__next__`, state variable, `StopIteration`. Verbose. 3. Show the generator equivalent — a function with `yield`. The interpreter does the protocol work: pauses on `yield`, resumes on `next()`, raises `StopIteration` when the function returns. Three lines instead of fifteen. 4. State the relationship explicitly: every generator is an iterator, not every iterator is a generator. Mention that generators also support `.send()`, `.throw()`, `.close()` — which plain iterators don't have unless you build them.

Edge cases to mention: - `__iter__` returning `self` is the convention for iterators; for *iterables* like `list`, `__iter__` returns a *new* iterator each time. That's why you can loop over a list twice but not over a generator twice (Q19). - A generator function that has `return value` raises `StopIteration(value)` — `value` becomes the iterator's "return value," accessible via `StopIteration.value` or via `yield from` (Q18). - Async generators (`async def` with `yield`) implement a separate protocol — `__aiter__` and `__anext__`. Easy to confuse with sync ones.

What NOT to say: - "Generators are lazy lists" — too loose; lists are sequences, generators are single-pass iterators. The semantic difference matters (Q19). - "They're the same thing" — they're not; iterator is the *protocol*, generator is *one way* to satisfy it.

Common follow-up: "Show me the same thing written as an iterator class and as a generator."

```
# Iterator class – manual protocol implementation
class Countdown:
    def __init__(self, start: int):
        self.n = start

    def __iter__(self):
        return self

    def __next__(self) -> int:
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1

# Generator – interpreter handles the protocol
def countdown(start: int):
    while start > 0:
        yield start
        start -= 1

print(list(Countdown(3)))    # [3, 2, 1]
print(list(countdown(3)))    # [3, 2, 1]
```

Stand-out line: *"Iterator is the protocol; generator is the syntactic sugar that lets you write the protocol in three lines of `yield` instead of fifteen lines of `__next__`."*

Q17 · Generator Expressions vs List Comprehensions

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Walmart Labs · Topic: Generators

The question (interviewer's verbatim phrasing):

When would you write `(x*2 for x in range(N))` instead of `[x*2 for x in range(N)]` ?

The 30-second answer: A list comprehension materialises every element in memory immediately — $O(N)$ space. A generator expression returns a generator object that produces elements lazily on demand — $O(1)$ space, but single-pass. Pick generator for huge or streaming data, or when you only consume the result once; pick list when you need to index, slice, length-check, or iterate multiple times.

Step-by-step thought process: 1. Lead with the memory point — that's the whole reason the language has both. List of 100M ints uses ~3.5 GB; generator uses ~200 bytes. The interviewer wants to see you internalise that. 2. State the consumption rule. List: random access, multiple iterations, `len()`, slicing — all free. Generator: forward-only, single-pass, no `len()`, no indexing. 3. Mention the syntactic detail: parentheses around a comprehension make a generator expression, except when it's the *sole argument* to a function call, where the parens are optional — `sum(x*2 for x in nums)` works without extra parens. 4. Anticipate the performance. Generators are usually a *touch* slower per element (function-call overhead per `next()`) but win massively on memory and on early termination. `any(...)`, `all(...)`, `next(...)` short-circuit and never build the full sequence.

Edge cases to mention: - `sum`, `min`, `max`, `any`, `all`, `next` all accept generators — use them and skip the intermediate list. - A generator captures variables by reference (late binding) — `[x for x in gen]` evaluates `gen` at iteration time, not at definition. Same closure-late-binding trap as lambdas (covered in File 05). - Don't pass a generator to a function that iterates over it twice — second pass is empty. List-ify with `list(...)` if you need that.

What NOT to say: - "Generator expressions are always better" — false for small data or anything needing indexing. - "They're the same with different brackets" — true syntactically, dangerously misleading semantically.

Common follow-up: "Why does `len(x for x in range(10))` raise an error?" — Generators don't have a known length without consuming them, so `len()` refuses. Use `sum(1 for _ in gen)` if you must count, accepting that it exhausts the generator.

```
import sys

# List: holds 1M ints
nums_list = [x * 2 for x in range(1_000_000)]
print(sys.getsizeof(nums_list))    # ~8.5 MB just for the list overhead

# Generator: holds nothing yet, materialises one at a time
nums_gen = (x * 2 for x in range(1_000_000))
print(sys.getsizeof(nums_gen))    # ~200 bytes

# Streaming sum — never builds the full sequence
total = sum(x * 2 for x in range(1_000_000))
print(total)
```

Stand-out line: "List comprehensions materialise; generator expressions stream. If you only need to walk the data once, parens beat brackets every time."

Q18 · `yield from` — Delegation and Exception Flow

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: Generators

The question (interviewer's verbatim phrasing):

What does `yield from` do? Is it just a shorthand for a `for` loop with `yield`, or is there more to it?

The 30-second answer: `yield from iterable` delegates iteration to a sub-iterable — pulling each value out and yielding it to the outer caller. For *plain* iterables it's equivalent to `for x in iterable: yield x`. For *sub-generators*, it's more — it forwards `.send()` values, propagates exceptions both ways, and captures the sub-generator's return value via `StopIteration.value`. The manual loop drops all of that.

Step-by-step thought process: 1. Start with the simple case. `yield from [1, 2, 3]` yields 1, then 2, then 3 to the outer caller. Same as the for-loop version. 2. Move to the interesting case — delegating to another generator. `yield from sub_gen()` makes the outer generator a transparent pipe. If the outer caller does `gen.send(value)`, that value goes to `sub_gen`, not the outer. If `sub_gen` raises, the exception bubbles through the outer generator's `yield from` line. If the outer caller throws into the outer generator (`gen.throw(...)`), it's forwarded to `sub_gen`. 3. State the return-value capture. `result = yield from sub_gen()` — when `sub_gen` returns (raises `StopIteration(value)`), `result` is bound to `value`. Manual `for x in sub_gen(): yield x` cannot recover the return value; it's silently dropped. 4. Mention the use case — flattening nested generators (recursive tree traversal), building generator pipelines, and (pre-asyncio) coroutine

chaining. Modern async code uses `await` for delegation; `yield from` survives in sync generator pipelines.

Edge cases to mention: - `yield from gen` advances `gen` to exhaustion before continuing in the outer generator — be careful with infinite sub-generators. - Exceptions raised *inside* `yield from` propagate to the *outer* caller of the outer generator, not to the consumer of the sub-generator — clean unwinding. - For sync iterables (lists, files, ranges), there's no practical difference vs the for-loop version; the magic is purely for sub-generators with `.send()` / `.throw()` / return values.

What NOT to say: - "It's just syntactic sugar for a for loop" — true for plain iterables, wrong for generator delegation. Always qualify. - "It's about async" — `yield from` predates `asyncio`; PEP 380 (Python 3.3) added it for delegation, not for async.

Common follow-up: "Write a recursive flatten that uses `yield from`."

```
from typing import Iterable, Any

def flatten(nested: Iterable[Any]):
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item)    # delegate to recursive call
        else:
            yield item

print(list(flatten([1, [2, [3, [4, 5]], 6], 7])))
# [1, 2, 3, 4, 5, 6, 7]

# Capturing a sub-generator's return value
def sub():
    yield 1
    yield 2
    return "done"

def outer():
    result = yield from sub()
    print(f"sub returned: {result}")
    yield 99

print(list(outer()))
# sub returned: done
# [1, 2, 99]
```

Stand-out line: "For plain iterables, `yield from` is a for-loop shortcut. For sub-generators, it's a delegation channel that forwards `.send()`, propagates exceptions, and captures the return value — three things the for-loop version silently drops."

Q19 · Iterator Exhaustion

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Generators

The question (interviewer's verbatim phrasing):

If I do `g = (x for x in range(5))` and then `list(g)` twice, what do I get?

The 30-second answer: First `list(g)` returns `[0, 1, 2, 3, 4]`. Second `list(g)` returns `[]`. Generators (and iterators in general) are single-pass — once `__next__` has raised `StopIteration`, it keeps raising. The generator object is "exhausted" and there's no rewind.

Step-by-step thought process: 1. Show the result first, then explain. Two `list(g)` calls; second one returns empty list, no error. That silent empty list is what bites people in production. 2. State the protocol reason. An iterator's `__iter__` returns `self`, not a fresh iterator. So `for x in g` resumes where the last loop left off — and the last loop left off at `StopIteration`. 3. Contrast with iterables. `list`, `dict`, `set`, `range` are *iterables* — their `__iter__` returns a brand-new iterator each time. You can loop over them as many times as you like. 4. Give the failure modes that bite real code. `zip(g, other)` consumes from `g` once. Passing `g` to a function that internally does `if not items: return; for x in items: ...` — the `if` consumes the first element via `bool(g)` -style misuse, then the for-loop sees the rest. File objects have the same property — `for line in f` once works; the second time you get nothing.

Edge cases to mention: - `itertools.tee(g, n)` gives you `n` independent iterators sharing the same source — useful when you genuinely need two passes, but it buffers everything between the slowest and fastest consumer. - If you really need to iterate twice, `data = list(g)` once and iterate the list. Trades memory for re-iterability. - `zip()` is itself a generator in Python 3 — same exhaustion behaviour. `zip(a, b)` is one-shot.

What NOT to say: - "You can reset a generator" — you can't. You can only create a fresh one by calling the generator function again. - "It raises an error the second time" — it doesn't; silent empty result. That's what makes it dangerous.

Common follow-up: "You wrote a function that takes an iterable and does two passes over it. A caller passes a generator and the second pass sees nothing. How do you fix the function?" — Either materialise inside (`data = list(items)`) and document the memory cost, or document that the function requires a re-iterable and not a one-shot iterator.

```

g = (x for x in range(5))
print(list(g))    # [0, 1, 2, 3, 4]
print(list(g))    # [] -- exhausted, no error

# Real-world bite: file objects
with open("data.txt", "w") as f:
    f.write("a\nb\nc\n")

with open("data.txt") as f:
    lines_count = sum(1 for _ in f)    # consumes the file
    first_line = next(iter(f), None)  # already exhausted
    print(lines_count, first_line)     # 3 None

# Iterable (list) – reusable
nums = [0, 1, 2, 3, 4]
print(list(nums))    # [0, 1, 2, 3, 4]
print(list(nums))    # [0, 1, 2, 3, 4]

```

Stand-out line: "Generators don't error on a second pass — they silently return empty. That silence is exactly why iterator exhaustion is the bug you find in staging at 11pm."

Q20 · `functools.cache` vs `lru_cache`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Walmart Labs · Topic: Decorators · Topic: Memoization

The question (interviewer's verbatim phrasing):

What's the difference between `@functools.cache` and `@functools.lru_cache`, and when would you reach for memoization in the first place?

The 30-second answer: Both decorators memoize a function — store `(args) -> result` in a dict so repeat calls with the same args skip the body. `lru_cache(maxsize=N)` bounds the cache and evicts the least-recently-used entry when full. `cache` (added in 3.9) is `lru_cache(maxsize=None)` — unbounded, slightly faster because it skips the LRU bookkeeping. Use `cache` for small input spaces (recursive maths); use `lru_cache(maxsize=N)` when inputs are unbounded but recent calls repeat.

Step-by-step thought process: 1. State the shared mechanism. The decorator wraps the function, builds a dict keyed by `args + tuple(sorted(kwargs.items()))`, returns the cached value if the key exists, otherwise calls the function and stores the result. 2. Spell out the difference.

`lru_cache(maxsize=128)` (the default) keeps at most 128 entries; on the 129th miss it evicts the least-recently-used. `cache` has no bound — entries live for the process lifetime. Memory cost grows

with unique inputs. 3. Mention `cache_info()` — both produce a wrapper with `.cache_info()` returning `(hits, misses, maxsize, currsize)`. Print it after a workload to see whether your cache is actually helping. 4. Anticipate the gotchas. (a) Args must be hashable — pass a `dict` and you get `TypeError`. (b) Methods on classes are problematic because `self` is part of the key — the whole instance gets pinned in the cache (memory leak). Use `functools.cached_property` for per-instance memoization instead. (c) Cache is per-process — won't survive a restart, won't cross workers.

Edge cases to mention: - `cache_clear()` empties the cache; useful in tests to avoid one test leaking into the next. - `lru_cache(typed=True)` keys distinguish `1` from `1.0` — relevant when numeric type affects the result. - For recursive functions, decorate *before* the function is defined so the recursive call hits the cache: `@cache; def fib(n): return n if n<2 else fib(n-1)+fib(n-2)` — naive $O(2^n)$ becomes $O(n)$.

What NOT to say: - "They're the same" — different in bounds and bookkeeping cost. - "Always use `cache`" — unbounded caches on user-facing functions are the textbook memory leak.

Common follow-up: *"Memoize Fibonacci and show me the cache stats."*

```
import functools

@functools.cache          # unbounded; same as lru_cache(maxsize=None)
def fib(n: int) -> int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)

fib(30)
print(fib.cache_info())
# CacheInfo(hits=28, misses=31, maxsize=None, currsize=31)

@functools.lru_cache(maxsize=128)  # bounded; evicts LRU when full
def fib_lru(n: int) -> int:
    return n if n < 2 else fib_lru(n - 1) + fib_lru(n - 2)

fib_lru(30)
print(fib_lru.cache_info())
# CacheInfo(hits=28, misses=31, maxsize=128, currsize=31)

fib.cache_clear()  # reset between test runs
```

Stand-out line: "`cache` is `lru_cache(maxsize=None)` minus the LRU bookkeeping — faster, unbounded, and the memory leak you'll regret on a long-running web worker."

Section 3 — OOP + dunder (10 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 01 Top 50

The ten OOP questions that consistently land in round 1 and round 2 across Indian product cos, GCCs, and Service MNCs. The interviewer is testing whether you've written Python classes for years or only inherited from `object` in college. Each Q ships in the Step-by-Step Answer format — 30-second answer, what to say out loud, edge cases, what NOT to say, follow-up. Code blocks compile on Python 3.12.

Q21 · `__init__` vs `__new__` — when each fires

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Goldman · Topic: OOP

The question (interviewer's verbatim phrasing):

What's the difference between `__init__` and `__new__` ? When would you actually override `__new__` ?

The 30-second answer: `__new__` is the constructor — it allocates the object and returns it. `__init__` is the initializer — it sets attributes on an already-built instance. `__new__` runs first, receives the class as its first argument, and you override it when you need to control allocation itself: singletons, instance caching, or subclassing immutable types like `int` , `str` , `tuple` .

Step-by-step thought process: 1. "Most candidates think `__init__` is the constructor. It isn't. `__init__` initializes an instance that already exists. The actual construction happens in `__new__` ." 2. " `__new__(cls, ...)` returns the new instance. Python then calls `__init__(self, ...)` on whatever `__new__` returned — but only if it's an instance of `cls` . If `__new__` returns something else, `__init__` is skipped." 3. "Day-to-day, I never override `__new__` because `object.__new__` does the right thing. I reach for it in three cases: singletons where I want one instance per class; instance caching like Python's small-int cache; and subclassing immutable types where `__init__` can't mutate the already-frozen instance." 4. "The immutable case is the clearest. If I subclass `tuple` or `str` , the instance is frozen by the time `__init__` runs — so I have to inject custom args during `__new__` ."

Code (Python 3.12):


```

class UpperStr(str):
    # str is immutable - __init__ can't set the value, so we shape it in __new__
    def __new__(cls, value: str) -> "UpperStr":
        return super().__new__(cls, value.upper())

x = UpperStr("razorpay")
print(x)          # RAZORPAY
print(type(x))    # <class '__main__.UpperStr'>

class Singleton:
    _instance = None

    def __new__(cls) -> "Singleton":
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self) -> None:
        # WARNING: __init__ runs on every Singleton() call, even when __new__
        # returns the cached instance. Guard mutations or use a flag.
        if not hasattr(self, "_ready"):
            self.connections = []
            self._ready = True

a = Singleton()
b = Singleton()
print(a is b)    # True

```

Edge cases to mention: - If `__new__` returns an object that is NOT an instance of `cls`, Python skips `__init__` entirely. This is how factory-style `__new__` works. - `__init__` on a singleton runs every time you call the class — so re-initialization is the real footgun, not allocation. Use a `_ready` flag or move all state-setup into `__new__`. - `__new__` is implicitly a static method (no `@staticmethod` needed) even though it takes `cls` first — Python special-cases it.

What NOT to say: - "`__init__` is the constructor." It isn't. Saying this on a senior round flags you instantly. - "You override `__new__` to make objects faster." False — allocation cost dominates only in micro-benchmarks; the reasons are semantic (singleton / immutable subclass / caching). - "`__new__` and `__init__` always run together." Not true — return a non-`cls` instance from `__new__` and `__init__` is skipped.

Common follow-up: "For a thread-safe singleton in Python, what would you change in this code?" — Wrap the `__new__` body in a `threading.Lock`, or use a module-level instance (modules are already singletons in CPython).

Q22 · MRO — C3 linearization and the diamond problem

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Cred, Flipkart · Topic: OOP

The question (interviewer's verbatim phrasing):

Explain Python's MRO. What is C3 linearization, and how does it handle the diamond problem?

The 30-second answer: MRO is the order Python walks the inheritance graph to resolve attribute and method lookups. Python 3 uses C3 linearization — a deterministic algorithm that preserves three rules: a class comes before its parents, parents appear in the order you wrote them in the class definition, and the order is monotonic across subclasses. You read it via `cls.__mro__` or `cls.mro()`.

Step-by-step thought process: 1. "Before I explain C3, I'll show why naive depth-first breaks. In a diamond — `D(B, C)`, where `B` and `C` both inherit from `A` — depth-first would visit `A` before `C`, meaning a method overridden in `C` gets silently bypassed. That's the diamond problem." 2. "C3 fixes this by merging the linearizations of all parents plus the parent order itself, picking the next class that doesn't appear later in any other list. The result satisfies all three constraints simultaneously." 3. "In practice I read `cls.__mro__` — it's a tuple — and that tells me the exact order `super()` will walk. For cooperative multiple inheritance, every class in the chain must call `super().method()` for the chain to actually thread through." 4. "If C3 can't find a consistent order — like `class X(A, B)` where one parent puts `A` before `B` and another puts `B` before `A` — Python raises `TypeError: Cannot create a consistent method resolution order` at class-definition time."

Code (Python 3.12):

```

class A:
    def who(self) -> str:
        return "A"

class B(A):
    def who(self) -> str:
        return "B -> " + super().who()

class C(A):
    def who(self) -> str:
        return "C -> " + super().who()

class D(B, C):
    def who(self) -> str:
        return "D -> " + super().who()

print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)

print(D().who())
# D -> B -> C -> A
# Notice: A runs ONCE, even though it's reachable via both B and C.

```

Edge cases to mention: - `object` is always last in the MRO. Every class implicitly inherits from it in Python 3. - The MRO is computed at class-creation time. Once the class exists, the MRO is frozen — patching parents later doesn't change the MRO of the subclass. - `super()` in Python 3 uses the MRO of the instance's actual class, not the lexical class. So `super().method()` inside `B` may resolve to `C` if the instance is a `D`.

What NOT to say: - "Python uses depth-first." Wrong for Python 3 — it's C3. Depth-first was the old-style-class MRO (Python 2.x with classic class hierarchy); C3 landed for new-style classes in 2.3 and is the only MRO in Python 3. - "MRO is left-to-right depth-first." Half right — it's left-to-right but linearized, not depth-first. The distinction is the diamond. - "I'd avoid multiple inheritance to dodge this." That's an opinion, not an answer. Mixins and ABCs are first-class Python patterns; the interviewer wants the mechanic.

Common follow-up: "What error do you see if you try to create a class with an inconsistent MRO?" — `TypeError: Cannot create a consistent method resolution order (MRO) for bases ...`, raised at class-definition time.

Q23 · `classmethod` vs `staticmethod` vs instance method

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: OOP

The question (interviewer's verbatim phrasing):

Walk me through `classmethod` vs `staticmethod` vs a regular instance method. When do you reach for each?

The 30-second answer: Instance methods take `self` and operate on an instance. `classmethod` takes `cls` and operates on the class — primary use is alternative constructors (`from_dict` , `from_csv`). `staticmethod` takes neither — it's a plain function living inside the class namespace for organizational reasons, no access to `cls` or `self` .

Step-by-step thought process: 1. "The deciding question is: what does this method need to do its job? If it needs the instance's state, it's an instance method. If it needs the class but not an instance — typically to build one — it's a `classmethod` . If it needs neither, it's a `staticmethod` ." 2. "The killer use case for `classmethod` is alternative constructors. Pandas does this with `DataFrame.from_dict` , `from_records` , `from_csv` . Each returns an instance of the class, and because they take `cls` , they automatically work on subclasses — call `from_dict` on a subclass and you get a subclass instance back." 3. " `staticmethod` is the least powerful of the three — it's just a function. Use it when the logic is conceptually tied to the class but doesn't need its state. If I'm reaching for `staticmethod` often, I check whether the function should just live at the module level." 4. "All three are descriptors — Python binds `self` or `cls` for you when you access them via an instance or class. That's why `classmethod` works correctly when called on subclasses."

Code (Python 3.12):

```

from datetime import datetime

class Order:
    def __init__(self, order_id: str, amount: float) -> None:
        self.order_id = order_id
        self.amount = amount

    # Instance method – needs self
    def with_gst(self) -> float:
        return self.amount * 1.18

    # Classmethod – alternative constructor, returns cls (subclass-friendly)
    @classmethod
    def from_payload(cls, payload: dict) -> "Order":
        return cls(order_id=payload["id"], amount=float(payload["amt"]))

    # Staticmethod – tied to the class conceptually, but no self/cls needed
    @staticmethod
    def is_business_hours(ts: datetime) -> bool:
        return 9 <= ts.hour < 21

class PriorityOrder(Order):
    pass

# from_payload on subclass returns subclass instance – that's the cls magic
p = PriorityOrder.from_payload({"id": "ORD-1", "amt": "499.00"})
print(type(p).__name__) # PriorityOrder
print(p.with_gst())     # 588.82
print(Order.is_business_hours(datetime.now()))

```

Edge cases to mention: - `classmethod` works correctly with inheritance because `cls` is the actual subclass at call time. Replace `cls(...)` with the explicit class name and you break that. - A `staticmethod` accessed through an instance still works — Python doesn't pass `self`. Same applies via the class. - You can stack `@classmethod` with `@property` only via the `__set_name__` / descriptor route on 3.9 — the short-lived `@classmethod @property` chain was deprecated and removed in 3.13.

What NOT to say: - "`staticmethod` is just for namespacing — use module functions instead." Sometimes true, but on a class with dozens of methods, grouping a helper under the class genuinely improves discoverability. - "`classmethod` is for class-level state mutation." That's an anti-pattern — class-level mutable state is shared across instances and tests. Use `classmethod` for construction. - "Instance methods are slower because of `self`." The cost is negligible in real code; pick the right semantic, not the micro-optimization.

Common follow-up: "Why is `cls` better than hard-coding the class name inside `from_payload`?" — Because `cls` resolves to the actual subclass at call time, so `PriorityOrder.from_payload(...)` returns a `PriorityOrder`, not an `Order`. Hard-coding kills subclass support.

Q24 · `__slots__` — what it does, what you lose

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Goldman, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

What does `__slots__` do? Why might you add it to a class, and what do you give up?

The 30-second answer: `__slots__` tells Python to store instance attributes in a fixed-size array rather than the per-instance `__dict__`. You get a meaningful memory drop and slightly faster attribute access. You give up the ability to add new attributes at runtime, the per-instance `__dict__` and `__weakref__` by default, and you have to be careful with inheritance — every class in the chain must declare `__slots__` for the savings to actually land.

Step-by-step thought process: 1. "By default every Python instance has a `__dict__` — a hash table mapping attribute name to value. Hash tables are flexible but expensive: a few hundred bytes per instance just for the dict, before any actual data." 2. "`__slots__ = ('order_id', 'amount')` tells Python: don't create a `__dict__`, store these two attributes in a fixed C array. Memory drops 40-60% for small attribute counts. Access is also marginally faster because we skip the hash lookup." 3. "The trade-offs land in three places. First, you can't set attributes not in `__slots__` — `AttributeError`. Second, no `__dict__` and no `__weakref__` unless you list them explicitly. Third, inheritance is fragile: if any class in the MRO skips `__slots__`, that class brings the `__dict__` back and you lose the memory win." 4. "Where this actually matters in production: when I'm holding millions of instances in memory — typical Indian data-platform use cases like Postman's request objects, Razorpay's transaction records, or anywhere I'm materializing a large query result into objects."

Code (Python 3.12):

```

class Order:
    __slots__ = ("order_id", "amount", "status")

    def __init__(self, order_id: str, amount: float, status: str) -> None:
        self.order_id = order_id
        self.amount = amount
        self.status = status

o = Order("ORD-1", 499.0, "PAID")
print(o.amount) # 499.0

try:
    o.refund_id = "RFND-1" # not in __slots__
except AttributeError as e:
    print(e) # 'Order' object has no attribute 'refund_id'

# Memory comparison
import sys

class Plain:
    def __init__(self, x: int, y: int) -> None:
        self.x, self.y = x, y

class Slotted:
    __slots__ = ("x", "y")
    def __init__(self, x: int, y: int) -> None:
        self.x, self.y = x, y

print(sys.getsizeof(Plain(1, 2)) + sys.getsizeof(Plain(1, 2).__dict__))
# ~336-344 bytes on Python 3.12+ (object + dict; dict grew between 3.7 and 3.12)
print(sys.getsizeof(Slotted(1, 2)))
# ~ 48 bytes

```

Edge cases to mention: - Inheriting from a class without `__slots__` quietly reintroduces `__dict__` in instances. Every class in the chain must declare `__slots__` (use `__slots__ = ()` on intermediate classes that add no new attributes). - `__slots__` is set in the class body — at runtime it becomes a descriptor on the class. Don't try to mutate it after class creation. - If you need weak references on slotted instances, add `"__weakref__"` to the tuple explicitly. Otherwise `weakref.ref(instance)` raises `TypeError`.

What NOT to say: - "`__slots__` makes everything faster." The speedup is ~10-15% on attribute access in tight loops; memory is the real win. - "I always use `__slots__`." Premature optimization. Use it on hot, high-cardinality classes. For a `Config` object instantiated once, it adds friction with no payoff. - "`__slots__` lets you do private attributes." No — that's name mangling with `__name`. Different mechanism.

Common follow-up: "What happens if I inherit from a slotted class and don't declare `__slots__` in the subclass?" — The subclass instance gets a `__dict__` back, killing the memory win. Always declare `__slots__ = ()` (or a tuple of new attrs) on every subclass.

Q25 · `__repr__` vs `__str__`**Tags:** Difficulty: Easy · Asked at: TCS, Infosys, Postman, Zerodha · Topic: OOP**The question (interviewer's verbatim phrasing):**

What's the difference between `__repr__` and `__str__` ? Which one does `print()` call, and which one does the REPL call?

The 30-second answer: `__repr__` is the unambiguous developer-facing representation — ideally eval-able so `eval(repr(obj))` reconstructs the object. `__str__` is the human-readable form. `print(obj)` and `str(obj)` call `__str__`; the interactive REPL and containers like `list` call `__repr__`. If you only define one, define `__repr__` — `__str__` falls back to it.

Step-by-step thought process: 1. "The mental model: `__repr__` is for me at 2am debugging a log. `__str__` is for the user. Different audiences, different goals." 2. "`__repr__` should be unambiguous. The convention is `ClassName(arg=value, arg=value)` — ideally something `eval` could turn back into an equal object. Python's built-ins follow this: `repr([1, 2])` is `'[1, 2]'`, which evals back to the same list." 3. "`__str__` is the friendly form. `str(datetime.now())` is `'2026-05-30 14:23:11.412'`; `repr(datetime.now())` is `'datetime.datetime(2026, 5, 30, 14, 23, 11, 412000)'`. Both useful, different jobs." 4. "Two routing rules to remember. One: `print(obj)` and f-strings without `!r` use `__str__`. Two: the interactive REPL when you type a bare expression and press Enter calls `__repr__` — that's also what happens when an object sits inside a list and you print the list. So a list of objects shows `__repr__` for each, not `__str__`."

Code (Python 3.12):


```

class Order:
    def __init__(self, order_id: str, amount: float) -> None:
        self.order_id = order_id
        self.amount = amount

    def __repr__(self) -> str:
        # eval-able, unambiguous
        return f"Order(order_id={self.order_id!r}, amount={self.amount!r})"

    def __str__(self) -> str:
        # human-friendly
        return f"Order #{self.order_id} - Rs. {self.amount:.2f}"

o = Order("ORD-42", 1499.0)

print(o)                # Order #ORD-42 - Rs. 1499.00      (calls __str__)
print(str(o))            # Order #ORD-42 - Rs. 1499.00      (calls __str__)
print(repr(o))           # Order(order_id='ORD-42', amount=1499.0)
print([o])               # [Order(order_id='ORD-42', amount=1499.0)] (list uses __repr__)
print(f"{o}")            # __str__
print(f"{o!r}")          # __repr__

# Defining only __repr__ makes __str__ fall back to it
class Bare:
    def __repr__(self) -> str:
        return "Bare()"

print(str(Bare()))      # Bare() - falls back to __repr__

```

Edge cases to mention: - If only `__repr__` is defined, `__str__` falls back to it. The reverse is NOT true — undefined `__repr__` gives you the default `<Order object at 0x...>`. - Logging frameworks default to `%s` (which calls `__str__`) but log records often use `%r` for arguments. Pick repr carefully if your objects show up in production logs. - `print([obj])` always calls `__repr__` on the element, never `__str__`. This trips up developers who define `__str__` and expect it to show in lists.

What NOT to say: - "They're the same thing." Different audiences, different conventions. Senior interviewers fail you on this immediately. - "I never define `__repr__`." Then your objects look like `<Order object at 0x7f...>` in every debugger and traceback. Always define at least `__repr__`. - "`__repr__` must be eval-able." Strong convention, not a hard rule — it's "ideally eval-able, otherwise unambiguous and informative." Sockets, file handles, threads can't be eval-able.

Common follow-up: "What's the difference between `f'{obj}'` and `f'{obj!r}'`?" — The `!r` flag forces `__repr__`. Bare interpolation uses `__str__`. There's also `!s` (explicit `__str__`) and `!a` (`ascii()` representation).

Q26 · `dataclasses.dataclass`**Tags:** Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Walmart Labs · Topic: OOP**The question (interviewer's verbatim phrasing):**

What does the `@dataclass` decorator actually generate? What's `frozen=True` for, and why do I need `field(default_factory=...)` ?

The 30-second answer: `@dataclass` reads the class annotations and auto-generates `__init__`, `__repr__`, `__eq__`, and optionally `__hash__` and ordering methods. `frozen=True` makes instances immutable and gives you a free `__hash__`. `field(default_factory=list)` is the correct way to default to a fresh mutable per instance — using `field=[]` directly hits the mutable-default-argument trap.

Step-by-step thought process: 1. "`@dataclass` is the answer to writing the same `__init__`, `__repr__`, `__eq__` boilerplate on every class. You declare attributes via class-level annotations, and the decorator generates the methods at class-creation time." 2. "`__eq__` is generated to compare field-by-field, so two dataclasses with the same field values compare equal. That's almost always what you want for value objects — DTOs, config records, parsed request payloads." 3. "`frozen=True` does two things: makes assignment raise `FrozenInstanceError`, and generates `__hash__` so you can use the instance in sets and as dict keys. Mutable dataclasses get `__hash__ = None`, which makes them unhashable — same rule as Q29." 4. "`field(default_factory=...)` exists because Python evaluates default arguments once at function-definition time. If you wrote `items: list[str] = []`, every dataclass instance would share that one list — the classic mutable-default bug. `default_factory=list` calls `list()` fresh for each instance."

Code (Python 3.12):

```

from dataclasses import dataclass, field, FrozenInstanceError
from typing import Any

@dataclass
class Order:
    order_id: str
    amount: float
    items: list[str] = field(default_factory=list) # fresh list per instance
    metadata: dict[str, Any] = field(default_factory=dict)

a = Order("ORD-1", 499.0)
b = Order("ORD-1", 499.0)
print(a == b)          # True - generated __eq__ compares field-by-field
print(a)               # Order(order_id='ORD-1', amount=499.0, items=[], metadata={})

a.items.append("shoe")
print(b.items)         # [] - fresh list per instance, not shared

@dataclass(frozen=True, slots=True)
class Money:
    amount: int # paise (avoid float for money - use int paise or Decimal)
    currency: str = "INR"

m = Money(49900)
print(hash(m))         # Works because frozen=True generated __hash__
try:
    m.amount = 100     # raises - frozen
except FrozenInstanceError as e:
    print(e)

# dataclass(kw_only=True) forces keyword arguments (Python 3.10+)
@dataclass(kw_only=True)
class PaymentRequest:
    merchant_id: str
    amount: int
    currency: str = "INR"

req = PaymentRequest(merchant_id="M-1", amount=49900)

```

Edge cases to mention: - `eq=True` is the default. If you set `eq=False`, `__hash__` is inherited from `object` (identity-based) — almost certainly not what you want. - `slots=True` (added in Python 3.10) gives you the `__slots__` memory win automatically. Useful but breaks if you also use `__post_init__` to set non-annotated attributes. - `field(repr=False)` excludes a field from `__repr__` — handy for secrets and tokens that shouldn't show in logs. - `__post_init__` runs after `__init__` and is the hook for validation or derived fields. Frozen dataclasses can't assign in `__post_init__` directly — use `object.__setattr__(self, ...)`.

What NOT to say: - "Dataclasses are slow." They're regular classes — the decorator runs once at class-definition time. Per-instance, they're identical to handwritten classes. - "Pydantic and dataclasses are interchangeable." They're not. Dataclasses don't validate types at runtime; Pydantic does. Pick by

whether you need runtime validation (Pydantic) or just boilerplate reduction (dataclasses). - "I'd default to `items: list = []` for brevity." This is the mutable-default trap — instances will share the list. Always `field(default_factory=list)`.

Common follow-up: "How would you make a frozen dataclass that also computes a derived field at construction?" — Use `__post_init__` with `object.__setattr__(self, "derived", value)` because direct assignment is blocked by the frozen state.

Q27 · Composition vs inheritance

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: OOP design

The question (interviewer's verbatim phrasing):

Why does the Gang of Four say "favor composition over inheritance"? Walk me through a real Python example where you'd pick composition.

The 30-second answer: Inheritance couples a subclass to its parent's entire surface area, including methods you don't want and changes you can't predict. Composition gives you a reference to a collaborator and you call exactly the methods you need — looser coupling, easier testing, no MRO surprises. Pick inheritance only when there's a genuine "is-a" relationship and you want to honor the parent's full contract; pick composition for "has-a" and for behavior you want to swap or mock.

Step-by-step thought process: 1. "The classic mistake: I'm building a `PaymentProcessor` and I see it needs an HTTP client, retry logic, and logging. I make `PaymentProcessor` inherit from `HttpClient` to get the requests for free. Now `PaymentProcessor` exposes every HTTP method to its callers, breaks LSP because it's not really an `HttpClient`, and I can't swap the HTTP client out for tests without monkey-patching." 2. "Composition fixes this. `PaymentProcessor` takes an `HttpClient` in `__init__` and delegates. Now I can pass a fake client in tests, swap to a different HTTP library, and the public API stays clean." 3. "Where inheritance still pays: when you genuinely have an is-a relationship and you'll honor LSP. ABCs are inheritance done right — `MutableMapping` defines a contract; my class promises to honor it. The contract is small and explicit. Or when you're extending a framework class whose extension point is subclassing." 4. "In Python specifically, mixins are a structured way to do narrow inheritance — small classes that add one capability. But even mixins should be small and orthogonal; if a mixin needs to know about its sibling, that's a signal to switch to composition."

Code (Python 3.12):

```

from typing import Protocol

# COMPOSITION – PaymentProcessor takes the collaborator
class HttpClient(Protocol):
    def post(self, url: str, json: dict) -> dict: ...

class PaymentProcessor:
    def __init__(self, http: HttpClient, merchant_id: str) -> None:
        self.http = http          # collaborator, not parent
        self.merchant_id = merchant_id

    def charge(self, amount_paise: int) -> dict:
        return self.http.post(
            "https://api.razorpay.com/v1/payments",
            json={"merchant": self.merchant_id, "amount": amount_paise},
        )

# Real HTTP client + a test double – both satisfy the Protocol
class RealHttp:
    def post(self, url: str, json: dict) -> dict:
        # imagine requests.post(...).json()
        return {"id": "pay_1", "status": "captured"}

class FakeHttp:
    def __init__(self) -> None:
        self.calls: list[tuple[str, dict]] = []
    def post(self, url: str, json: dict) -> dict:
        self.calls.append((url, json))
        return {"id": "pay_fake", "status": "captured"}

# Test setup is trivial – just inject the fake
fake = FakeHttp()
proc = PaymentProcessor(http=fake, merchant_id="M-1")
proc.charge(49900)
assert fake.calls[0][1]["amount"] == 49900

# INHERITANCE – same logic, way worse
class PaymentProcessorBad(RealHttp): # PaymentProcessor IS-A HttpClient? No.
    def __init__(self, merchant_id: str) -> None:
        super().__init__()
        self.merchant_id = merchant_id
    # Now .post() is part of PaymentProcessorBad's public API. Yuck.

```

Edge cases to mention: - Composition lets each collaborator have its own lifecycle, configuration, and tests. Inheritance tangles them. - For framework extension points (Django generic views, FastAPI dependencies), inheritance is the idiom — don't fight the framework. - Dependency injection via `__init__` is composition's best form in Python — no DI framework required, just constructor arguments.

What NOT to say: - "Never use inheritance." Wrong — ABCs, exceptions, dataclasses, and framework hooks all use inheritance correctly. - "Composition is always slower because of the extra indirection." The indirection is a single attribute lookup. Performance is not the axis on which to make this choice. - "Composition is just inheritance with more typing." Misses the LSP and surface-area points. Composition keeps your public API small.

Common follow-up: "How would you write the same `PaymentProcessor` test if it used inheritance instead of composition?" — You'd have to monkey-patch the inherited `post` method or build a fake subclass — both fragile and harder to read than constructor injection.

Q28 · ABCs vs `typing.Protocol` — nominal vs structural

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Goldman, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

What's the difference between an ABC and a `typing.Protocol` ? When would you choose each?

The 30-second answer: ABCs (from `abc`) are nominal — a class must explicitly inherit from the ABC to be recognized as a subtype, and `@abstractmethod` blocks instantiation until the methods are implemented. Protocols (from `typing`) are structural — any class with the right methods satisfies the protocol, no inheritance needed, like Go interfaces. Reach for ABCs when you control all implementers and want to enforce the contract at instantiation; reach for Protocols when you want duck typing with static checking.

Step-by-step thought process: 1. "Nominal subtyping means the relationship is declared — `class Razorpay(PaymentGateway)`. Structural subtyping means the relationship is inferred from shape — any class with `.charge(amount: int) -> dict` satisfies a `PaymentGateway` Protocol whether or not it's declared." 2. "ABCs are stricter and more discoverable. `class MyMap(MutableMapping)` forces you to implement the abstract methods; `MyMap()` raises `TypeError` until you do. The cost: every implementer needs to know about the ABC and inherit from it. Fine when you own the code." 3. "Protocols are looser and friendlier to code you don't control. Third-party clients can satisfy your Protocol without importing it — useful when integrating with libraries like `requests`, `httpx`, or a customer's adapter." 4. "The decision in practice: am I defining a public extension point that third parties will implement against (Protocol — they shouldn't have to inherit from my class)? Or am I enforcing a contract on internal subclasses I write (ABC — the IDE and `mypy` should flag missing methods)? Both can coexist — Protocols for the outer boundary, ABCs for internal hierarchies."

Code (Python 3.12):

```

from abc import ABC, abstractmethod
from typing import Protocol, runtime_checkable

# ABC – nominal. Implementers must inherit.
class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, amount_paise: int) -> dict: ...
    @abstractmethod
    def refund(self, payment_id: str) -> dict: ...

class Razorpay(PaymentGateway):
    def charge(self, amount_paise: int) -> dict:
        return {"id": "pay_1", "status": "captured"}
    def refund(self, payment_id: str) -> dict:
        return {"id": "rfnd_1", "status": "processed"}

# Forgetting an abstract method blocks instantiation
class Broken(PaymentGateway):
    def charge(self, amount_paise: int) -> dict:
        return {}
try:
    Broken()
except TypeError as e:
    print(e) # Can't instantiate abstract class Broken without an implementation for ...

# Protocol – structural. No inheritance needed.
@runtime_checkable
class Notifier(Protocol):
    def send(self, to: str, body: str) -> None: ...

class SmsNotifier:
    # does NOT inherit from Notifier
    def send(self, to: str, body: str) -> None:
        print(f"SMS to {to}: {body}")

class EmailNotifier:
    # also does NOT inherit
    def send(self, to: str, body: str) -> None:
        print(f"Email to {to}: {body}")

def notify(n: Notifier, to: str, body: str) -> None:
    n.send(to, body)

notify(SmsNotifier(), "+91-9876543210", "OTP: 4242")
notify(EmailNotifier(), "user@example.com", "Receipt attached")
print(isinstance(SmsNotifier(), Notifier)) # True – runtime_checkable allows this

```

Edge cases to mention: - `@runtime_checkable` on a Protocol enables `isinstance()` checks at runtime, but only verifies attribute presence — not signatures. Static checkers like `mypy` are stricter. - ABCs can also do structural-ish checks via `__subclasshook__`, but it's awkward — Protocols are the modern path. - A class can satisfy multiple Protocols implicitly. With ABCs you'd write `class X(A, B, C)` — Protocols are zero-cost. - ABCs may include concrete methods alongside abstract ones (mixins). Protocols generally shouldn't — they're contracts, not implementations.

What NOT to say: - "Protocols replace ABCs." They coexist. Standard library still uses ABCs heavily (`collections.abc`) and they're right for those cases. - "Protocols are just type hints, they don't do anything at runtime." Half true — `@runtime_checkable` makes `isinstance()` work. And static type checkers enforce them long before runtime. - "ABCs are slower because of `isinstance` overhead." Negligible. Pick by semantics, not micro-perf.

Common follow-up: "You're writing a plugin system. ABC or Protocol?" — Depends on who writes plugins. If it's first-party code, ABC gives you discoverability and instantiation-time enforcement. If third parties write plugins in their own packages, Protocol lets them satisfy your interface without depending on your module.

Q29 · `__eq__` and `__hash__` — the contract

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Razorpay, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

If I define `__eq__` on a class, what happens to `__hash__` ? Why can't mutable objects be hashable?

The 30-second answer: Defining `__eq__` on a class sets `__hash__` to `None` automatically — the instance becomes unhashable. The contract is: if `a == b` then `hash(a) == hash(b)`, forever. Mutable objects can change their equality basis at runtime, so their hash would change too — which would break every dict and set holding the instance. Lists, dicts, and sets are unhashable for exactly this reason.

Step-by-step thought process: 1. "Two rules. One: equal objects must hash equal. Two: an object's hash must not change over its lifetime. Together these mean a hashable object's equality basis has to be immutable." 2. "When you define `__eq__`, Python doesn't know how you're comparing — but it does know the default `__hash__` (which hashes by `id()`) almost certainly violates rule one. So Python sets `__hash__ = None`, making the instance unhashable. If you want it hashable, you must define `__hash__` explicitly." 3. "For an immutable value type — say, `Money(amount, currency)` — the right move is define `__eq__` and `__hash__` together over the same fields. Or use `@dataclass(frozen=True)`, which generates both for you and refuses mutation." 4. "For mutable types you want to keep unhashable. Lists, dicts, sets all follow this rule. If you put a list in a set and then mutate the list, the set's internal bucket pointer is stale — lookups fail unpredictably. Hence the `TypeError: unhashable type: 'list'`."

Code (Python 3.12):


```

class Money:
    def __init__(self, amount: int, currency: str = "INR") -> None:
        self.amount = amount
        self.currency = currency

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Money):
            return NotImplemented
        return self.amount == other.amount and self.currency == other.currency

    # Without this, Money becomes unhashable the moment __eq__ is defined
    def __hash__(self) -> int:
        return hash((self.amount, self.currency))

a = Money(49900)
b = Money(49900)
print(a == b)           # True
print(hash(a) == hash(b)) # True - contract holds
s = {a, b}
print(len(s))           # 1 - they hash and compare equal

# Mutable class WITHOUT defining __hash__ - Python sets it to None
class Cart:
    def __init__(self, items: list[str]) -> None:
        self.items = items

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Cart):
            return NotImplemented
        return self.items == other.items

try:
    {Cart(["shoe"])}
except TypeError as e:
    print(e) # unhashable type: 'Cart'

# To explicitly mark unhashable
class Explicit:
    __hash__ = None
    def __eq__(self, other: object) -> bool:
        return False

```

Edge cases to mention: - Returning `NotImplemented` from `__eq__` (not `False`) lets Python try the other operand's `__eq__` — required for proper subclass equality. - `frozenset` is hashable because it's immutable; `set` is not. Same for `tuple` vs `list`. The pattern repeats across the language. - Inheriting from a class with `__eq__` but no `__hash__`: the subclass also has `__hash__ = None`. You have to redefine both if you want the subclass hashable. - `@dataclass(frozen=True)` generates both `__eq__` and `__hash__` based on field values. `@dataclass(frozen=False)` generates only `__eq__` and sets `__hash__` to `None`.

What NOT to say: - "I'll just return `id(self)` from `__hash__`." Breaks the contract — `a == b` but `hash(a) != hash(b)`, so sets and dicts misbehave. - "Defining `__eq__` alone is fine." Only if you never put the instance in a set or dict key — which is rarely the case in real code, and the failure mode is silent: the instance becomes unhashable. - "Tuples are hashable because they're tuples." They're hashable because they're immutable AND their contents are hashable. `(1, [2])` is NOT hashable because the inner list isn't.

Common follow-up: "What does returning `NotImplemented` from `__eq__` do that returning `False` doesn't?" — `NotImplemented` tells Python "I can't compare these types; try the other operand's `__eq__`." Returning `False` short-circuits that, breaking symmetry — `a == b` might be `False` while `b == a` is `True`.

Q30 · `@property` — when to reach for it

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: OOP

The question (interviewer's verbatim phrasing):

What does `@property` do? Walk me through getter, setter, and deleter. When would you use it over a plain attribute?

The 30-second answer: `@property` turns a method into a computed attribute — callers access `obj.amount` instead of `obj.amount()`. You add `@amount.setter` and `@amount.deleter` to control writes and deletion. Reach for it when you start with a plain attribute and later need validation, derivation, or to deprecate it without breaking callers — the API stays attribute-style.

Step-by-step thought process: 1. "The Python convention is: start with a plain attribute. `self.amount = amount` in `__init__`, done. Don't write Java-style `get_amount()` / `set_amount()` getters by default — that's noise." 2. "Reach for `@property` when something changes. You need to validate writes (`amount` must be positive). You need to compute a value on read (`amount_with_gst`). You need to migrate callers from a now-derived field. The win is the public API doesn't change — `obj.amount` still works." 3. "Three decorators work together. `@property` defines the getter. `@amount.setter` defines the setter, called on `obj.amount = x`. `@amount.deleter` defines what happens on `del obj.amount`. The setter and deleter use the property name, not `property`." 4. "One gotcha: properties only work on classes, not on instances. So `obj.amount = property(...)` doesn't do what you'd hope. Define them in the class body."

Code (Python 3.12):

```

class Order:
    def __init__(self, amount_paise: int) -> None:
        self._amount = amount_paise    # underscore prefix = "use the property"

    @property
    def amount(self) -> int:
        """Amount in paise."""
        return self._amount

    @amount.setter
    def amount(self, value: int) -> None:
        if value < 0:
            raise ValueError(f"amount must be non-negative, got {value}")
        self._amount = value

    @amount.deleter
    def amount(self) -> None:
        # Logging deletion is a common real use – audit trail
        print(f"Order amount {self._amount} cleared")
        del self._amount

    # Derived property – no setter, read-only
    @property
    def amount_with_gst(self) -> float:
        return self._amount * 1.18

o = Order(49900)
print(o.amount)           # 49900    – looks like an attribute
print(o.amount_with_gst)  # 58882.0 – computed on read

o.amount = 100000         # goes through setter, validates
try:
    o.amount = -1
except ValueError as e:
    print(e)               # amount must be non-negative, got -1

del o.amount              # goes through deleter

# Read-only enforcement: omit the setter
class Stamp:
    def __init__(self, value: str) -> None:
        self._value = value
    @property
    def value(self) -> str:
        return self._value

s = Stamp("approved")
try:
    s.value = "rejected"
except AttributeError as e:
    print(e) # property 'value' of 'Stamp' object has no setter

```

Edge cases to mention: - Properties live on the class, not the instance. `obj.__dict__` shows `_amount`, not `amount`. Same applies to `vars(obj)`. - A property with only `@property` (no setter) is read-only. Assignment raises

`AttributeError: property 'X' of 'Y' object has no setter` in Python 3.11+. - Properties don't pickle the computed value — only the underlying state. Means side effects in getters don't fire on unpickle, which is usually what you want. - `@cached_property` from `functools` is the lazy variant — computes once, stores in the instance `__dict__`, returns the cached value on subsequent reads. Doesn't work on slotted classes without `"__dict__"` in `__slots__`.

What NOT to say: - "I always use `@property` for getters and setters." That's Java-style ceremony in Python. Start with plain attributes; promote to property when you need behavior. - "`@property` is faster than method calls." It's a method call wrapped in a descriptor — same cost, plus a tiny descriptor lookup. - "Properties replace `__slots__`." Different mechanisms. Properties control access; `__slots__` controls storage. They compose.

Common follow-up: "What's the difference between `@property` and `@cached_property`?" — `@property` recomputes on every access. `@cached_property` (from `functools`) computes once per instance, stores in `__dict__`, and serves from cache thereafter. Use `cached_property` for expensive derivations that don't change after first access; use `property` when the underlying state can change.

Section 4 — Exceptions, Typing, Modules

Ten questions on how Python handles errors and how modern code declares intent. These show up in nearly every backend round and a fair number of data rounds — the EAFP question alone has saved candidates from sounding like they wrote Java for ten years and Python for two months. Typing questions have grown sharper since 3.10 made union syntax cleaner, so the interviewer is testing whether you've updated your habits. Module questions catch out experienced devs who've never had to debug a circular import in anger.

Q31 · EAFP vs LBYL

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Freshworks, Zoho · Topic: Exceptions

The question (interviewer's verbatim phrasing):

What does EAFP mean in Python? When would you still prefer LBYL?

The 30-second answer: EAFP — "easier to ask forgiveness than permission" — is the Pythonic idiom of attempting the operation inside a `try` and catching the exception if it fails. LBYL — "look before you leap" — checks preconditions first. EAFP is preferred in Python because it's atomic (no race conditions), faster on the happy path, and reads cleaner; LBYL still wins when the check is cheap and the failure is genuinely expected, not exceptional.

Step-by-step thought process: 1. "EAFP means I just do the thing and catch the exception if it fails. LBYL means I check first with an `if` statement and then do the thing only if the check passes." 2. "The classic example is dict access. LBYL is `if 'key' in d: value = d['key']` — two lookups. EAFP is `try: value = d['key'] except KeyError: ...` — one lookup, atomic." 3. "Atomicity matters when something else can change state between the check and the use. With LBYL on a file — `if os.path.exists(path): open(path)` — another process can delete the file between those two lines. EAFP avoids the TOCTOU race." 4. "Performance-wise, exceptions in Python are cheap when not raised and expensive when raised. So EAFP wins when failure is rare. If you expect failure half the time, LBYL with a cheap check can be faster." 5. "I still reach for LBYL when the check is essentially free and the cost of the operation is high — for example, checking a list is non-empty before calling a heavy database write."

Edge cases to mention: - `dict.get(key, default)` is a third option that sidesteps the debate entirely for dict access — use it when a default makes sense. - For file paths, `pathlib.Path.exists()` followed by `.open()` is LBYL with a race window; just `try: open(path) except FileNotFoundError:` is the EAFP form. - Try/except is fast when no exception fires, but the cost of catching is non-trivial. Don't wrap a million-iteration loop in a try expecting most iterations to raise.

What NOT to say: - "EAFP is always better." It's the default Python style but LBYL has a place — saying always-better signals you parrot blog posts. - "Exceptions in Python are slow so I avoid them." Half-right and half-wrong. Raising is expensive; the try block itself is nearly free.

Common follow-up: "Show me a case where EAFP avoids a bug that LBYL would have." — File existence check then open is the cleanest one; checking a key in a dict that another thread is mutating is another.

```
# LBYL – two lookups, race-prone for shared dicts
if "user_id" in payload:
    user = payload["user_id"]
else:
    user = None

# EAFP – one lookup, atomic, Pythonic
try:
    user = payload["user_id"]
except KeyError:
    user = None

# Best for this specific case – dict.get
user = payload.get("user_id")
```

Q32 · try / except / else / finally

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: Exceptions

The question (interviewer's verbatim phrasing):

Walk me through the four clauses — try, except, else, finally. What runs when?

The 30-second answer: `try` runs the protected code. `except` runs only if a matching exception was raised. `else` runs only if no exception was raised — it's the "happy path continuation." `finally` runs always, exception or not, even if you `return` from inside the try.

Step-by-step thought process: 1. "The `try` block runs first. If it completes without raising, control goes to `else` if present, then `finally` ." 2. "If an exception is raised, Python checks each `except` clause in order. The first one whose class matches catches it. After the `except` runs, control goes to `finally` — `else` is skipped." 3. " `else` exists so you can separate 'the risky line' from 'the lines that depend on it succeeding.' The point is to keep the try block minimal — only the line that can raise — so you don't accidentally catch exceptions from the success path." 4. " `finally` always runs. Even if the try has a `return` , even if an `except` has a `return` , even if an exception propagates up because nothing caught it. That makes finally the place for resource cleanup — closing files, releasing locks, rolling back transactions." 5. "Modern Python — use a context manager (`with`) instead of try/finally for resource cleanup wherever you can. The context manager is the cleaner idiom; try/finally is for cases where no context manager fits."

Edge cases to mention: - If `finally` itself contains a `return` , it overrides any return or exception from the try/except blocks. Almost always a bug — don't put control flow in finally. - An exception raised in `except` or `else` is also caught by `finally` . So finally can run even when a second exception is propagating. - If the `try` block raises an exception type that no `except` matches, the exception propagates after `finally` runs.

What NOT to say: - " `else` runs at the end of try." Wrong — else runs only if no exception. Confusing else with finally is the classic mistake. - " `finally` doesn't run if you return from try." Wrong — finally runs even on return.

Common follow-up: "Why would you put a line in `else` instead of just at the end of `try` ?" — Because lines at the end of try can themselves raise, and you don't want the except to accidentally catch errors from the success-path logic.

```
import json

def load_config(path: str) -> dict:
    try:
        f = open(path, encoding="utf-8")
    except FileNotFoundError:
        return {} # missing config is fine, use defaults
    else:
        # only runs if open succeeded – parse errors stay uncaught here
        data = json.load(f)
        return data
    finally:
        # runs whether open succeeded or not; guard for the failed case
        if "f" in locals():
            f.close()
```

Q33 · Custom Exceptions — When and How

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Freshworks · Topic: Exceptions

The question (interviewer's verbatim phrasing):

When do you define a custom exception, and what do you inherit from?

The 30-second answer: Define a custom exception when callers need to distinguish your failure mode from a generic one — they want to write `except PaymentDeclined` not `except Exception`. Inherit from `Exception`, never from `BaseException`. `BaseException` is reserved for system-level interruptions like `KeyboardInterrupt` and `SystemExit` that user code shouldn't accidentally swallow.

Step-by-step thought process: 1. "The rule of thumb — if I'm raising it, callers will want to catch it specifically, so it needs its own class." 2. "I create a base exception for the module or package — say `PaymentError(Exception)` — and then specific subclasses for each failure mode — `PaymentDeclined(PaymentError)`, `PaymentTimedOut(PaymentError)`, `InvalidCardNumber(PaymentError)`. That way callers can catch the whole family or pick one." 3. "Always `Exception`, not `BaseException`. `BaseException` is the root and includes `KeyboardInterrupt`, `SystemExit`, and `GeneratorExit`. If you inherit from `BaseException`, a generic `except Exception` won't catch yours, but you also risk being caught by code that's trying to catch Ctrl-C cleanly. Just don't." 4. "Add useful state in `__init__` — error code, the input that failed, the upstream response — and override `__str__` for the human-readable message. Future-you debugging at 2 a.m. will thank current-you." 5. "Don't over-engineer. If the standard library already has the right exception — `ValueError`, `TypeError`, `KeyError`, `FileNotFoundError` — just raise that. Custom exceptions are for domain concepts the `stdlib` doesn't model."

Edge cases to mention: - Inheriting from a built-in like `ValueError` is fine when your exception genuinely is a value error — `class InvalidPincode(ValueError)` reads naturally and callers catching `ValueError` still get it. - The empty-body `class Foo(Exception): pass` pattern is normal — you don't need methods if the class is just a tag. - Exception classes are conventionally suffixed with `Error` — `PaymentError`, not `PaymentException`.

What NOT to say: - "Inherit from `BaseException` so it's harder to accidentally catch." Backwards reasoning — you want callers to be able to catch your exception with a broad `except Exception`. - "Custom exceptions are overkill, just use `ValueError` everywhere." Loses information at the catch site. Callers can't distinguish your failure from any other `ValueError` in the stack.

Common follow-up: "How would you let the caller distinguish a transient failure from a permanent one?" — Two subclasses of your base — `TransientPaymentError` and `PermanentPaymentError`. Callers can catch one and retry, let the other propagate.

```
class PaymentError(Exception):
    """Base for all payment-module failures."""

class PaymentDeclined(PaymentError):
    def __init__(self, reason_code: str, gateway_message: str):
        super().__init__(f"declined: {reason_code} - {gateway_message}")
        self.reason_code = reason_code
        self.gateway_message = gateway_message

class PaymentTimedOut(PaymentError):
    """Network timed out; safe to retry."""

# Caller can catch the family or a specific failure
try:
    charge_card(order)
except PaymentDeclined as e:
    log.warning("declined %s", e.reason_code)
except PaymentTimedOut:
    schedule_retry(order)
```

Q34 · Exception Chaining — raise X from Y

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Postman, Goldman · Topic: Exceptions

The question (interviewer's verbatim phrasing):

What does `raise X from Y` do? And what's the difference between `__cause__` and `__context__`?

The 30-second answer: `raise X from Y` declares that exception Y caused exception X — it sets `__cause__ = Y` and prints "The above exception was the direct cause of the following exception" in the traceback. `__context__` is set automatically whenever an exception is raised while another is being handled; it prints "During handling of the above exception, another exception occurred." Use `from` to make the chain intentional; use `from None` to suppress the chain entirely.

Step-by-step thought process: 1. "There are three states. Plain `raise X` from inside an except block — Python implicitly sets `X.__context__ = Y`, the old exception you were handling. Traceback shows 'During handling...'. " 2. "`raise X from Y` — explicit chaining. Sets `__cause__ = Y` and also implicitly suppresses the context. Traceback shows 'direct cause'. Use this when you're translating a low-level exception into a domain-level one." 3. "`raise X from None` — suppresses the chain. Use when the underlying exception is noise — say your library wraps a low-level encoding error into a clean `ConfigError` and the user doesn't need to see the binascii internals." 4. "Practical pattern — wrap library exceptions at API boundaries. The caller of my `payments` module should see `PaymentError`, not `requests.exceptions.ConnectionError`. So I catch the low-level error and `raise PaymentTimeout(...) from e` so the original is still available for debugging via `e.__cause__`." 5. "`__cause__` is set deliberately; `__context__` is set automatically. Both show in the traceback unless suppressed. When debugging, check both — the most useful info often hides in `__context__` when nobody used `from`."

Edge cases to mention: - `from None` doesn't actually unset `__context__` — it just sets `__suppress_context__ = True`, which hides it from the traceback. The original is still on the exception object if you really need it. - A bare `raise` re-raises the current exception with its original traceback. `raise X` raises a new one. Don't confuse them. - Chained exceptions can chain deeply — $Y \rightarrow X \rightarrow Z$. The traceback walks the chain backwards from the outermost.

What NOT to say: - "`from` is just for the traceback printout." Misses that `__cause__` is a real attribute you can inspect in code. - "I'd just log the original and raise a new one." Loses the chain — `from e` does the right thing automatically.

Common follow-up: "Why would you ever use `from None`?" — When the underlying exception leaks implementation details the caller shouldn't see (encoding errors from internal serialization, ORM internals at an HTTP-layer boundary).

```
def load_config(path: str) -> dict:
    try:
        with open(path) as f:
            return json.loads(f.read())
    except FileNotFoundError as e:
        # Domain exception, but original preserved via __cause__
        raise ConfigMissing(f"no config at {path}") from e
    except json.JSONDecodeError as e:
        # Hide the JSON internals from callers — they just want "bad config"
        raise ConfigInvalid(f"config at {path} is not valid JSON") from None
```

Q35 · ExceptionGroup and except*

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Exceptions

The question (interviewer's verbatim phrasing):

What's `ExceptionGroup` and the `except*` syntax? When have you used it?

The 30-second answer: `ExceptionGroup` (Python 3.11+) bundles multiple exceptions into one — used when several operations fail concurrently and you want to report all the failures, not just the first one. `except*` (note the star) catches matching exceptions out of the group while letting the rest re-raise. The classic use case is `asyncio.TaskGroup`, which collects every task failure into one `ExceptionGroup` instead of swallowing all but one.

Step-by-step thought process: 1. "Before 3.11, if you ran ten async tasks in parallel and three failed, you saw one exception and the others vanished. `ExceptionGroup` fixes that — `TaskGroup` raises an `ExceptionGroup` containing all three." 2. "You catch from a group with `except*`. `except* ValueError` catches every `ValueError` in the group and leaves any non-`ValueError` exceptions in a new group that propagates. So you can handle one failure mode and let the rest bubble up — without losing them." 3. " `except*` always gives you a group, even if only one exception matched. So the handler receives `eg: ExceptionGroup[ValueError]`, not a single `ValueError`. Iterate `eg.exceptions` to see each one." 4. "You can construct groups manually too — `raise ExceptionGroup('multiple failures', [err1, err2])`. Useful in a custom retry loop where you want to report every attempt's failure." 5. "As of 2026 interviewers ask this because `asyncio.TaskGroup` is the modern way to run concurrent tasks. If you say 'I use `asyncio.gather`' without mentioning `TaskGroup` or `ExceptionGroup`, you're answering with 2021 Python."

Edge cases to mention: - `BaseExceptionGroup` is the parent class; `ExceptionGroup` is the subclass that only holds non-`BaseException` exceptions. Catch the right one for what you raise. - `except*` cannot be mixed with plain `except` in the same try block — pick one syntax style per try. - A group's `.split(predicate)` method returns `(matching, rest)` — useful when you want richer logic than `except*` allows.

What NOT to say: - "ExceptionGroup is just a list of exceptions." Technically true, but misses that it's a real exception itself, can be raised, and integrates with `except*`. - "I just use `asyncio.gather(return_exceptions=True)`." That's the pre-3.11 workaround and still has its place, but `TaskGroup` + `ExceptionGroup` is the modern path interviewers want to hear.

Common follow-up: "What does `except* ValueError as eg` give you when the group has a `ValueError` and a `TypeError`?" — `eg` is an `ExceptionGroup[ValueError]` containing just the `ValueError`; the `TypeError` is in a separate group that re-raises automatically after the `except` block.

```
import asyncio

async def fetch(url: str) -> str:
    ... # raises ConnectionError or TimeoutError

async def main():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(fetch("https://razorpay.com/api/x"))
            tg.create_task(fetch("https://razorpay.com/api/y"))
            tg.create_task(fetch("https://razorpay.com/api/z"))
    except* ConnectionError as eg:
        for err in eg.exceptions:
            log.warning("conn failed: %s", err)
    except* TimeoutError as eg:
        log.error("timed out: %d tasks", len(eg.exceptions))
```

Q36 · Optional[X] vs X | None

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Freshworks, Swiggy · Topic: Typing

The question (interviewer's verbatim phrasing):

What's the difference between `Optional[int]` and `int | None` ? Which should I use in new code?

The 30-second answer: They mean exactly the same thing — both declare "this value is an int or None." `Optional[X]` is the older syntax from `typing` ; `X | None` is the modern syntax added in 3.10 via PEP 604. In any new 3.10+ codebase use `X | None` — it's the same characters or fewer, no import needed, and reads more like a union type than a special case.

Step-by-step thought process: 1. "Both are 'value is X or None.' Type checkers treat them identically — `Optional[int]` desugars to `Union[int, None]`, which is `int | None` in modern syntax." 2. "The older form needs `from typing import Optional`. The newer form is a built-in operator on type objects — no import." 3. "For new code on Python 3.10+, prefer `int | None`. Reads naturally, matches how you'd say it out loud — 'int or None.'" 4. "There's also a subtle anti-pattern worth flagging — `Optional[X]` does not mean 'X with a default of None.' It just means 'value can be None.' A parameter `def f(x: int | None)` still requires the caller to pass something; if you want it optional with a None default you also need `= None`." 5. "In code that still supports 3.8 or 3.9, you're stuck with `Optional` or you add `from __future__ import annotations` and use the new syntax for annotations only — they're evaluated as strings at module load."

Edge cases to mention: - `Optional[X]` evaluated at runtime gives `typing.Optional[X]`, which is `typing.Union[X, None]`. The `X | None` form gives `types.UnionType` — different concrete type, same semantic meaning. Matters only for runtime introspection. - For dataclasses with mutable defaults, `field: list[int] | None = None` is the safe pattern — never `field: list[int] = []`. - A function parameter `x: int | None` with no default is still required. Defaults are independent of types.

What NOT to say: - "`Optional[int]` means the argument is optional." Wrong — that's the trap. Optional types are about None-ability, not parameter optionality. - "They behave differently." They don't — for type-check purposes they're identical.

Common follow-up: "What about `Optional` versus a default of `None` ? Same thing?" — No — Optional types the value, default lets the caller omit it. Independent decisions.

```
# Modern 3.10+ — preferred in new code
def find_user(user_id: int) -> User | None:
    return db.users.get(user_id)

# Old form — still valid, needed for < 3.10
from typing import Optional
def find_user_legacy(user_id: int) -> Optional[User]:
    return db.users.get(user_id)

# Independence of nullability and defaultness
def update(user_id: int, name: str | None = None) -> None:
    # name is "either a string or None" AND defaults to None
    ...
```

Q37 · Union[X, Y] vs X | Y

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Typing

The question (interviewer's verbatim phrasing):

Same question for unions — `Union[int, str]` versus `int | str`. When does the new syntax not work?

The 30-second answer: Same story as Optional — `int | str` (PEP 604, 3.10+) is the modern equivalent of `Union[int, str]`. The only place the old syntax still wins is when you need to use it as a string annotation in old code without `from __future__ import annotations`, or when you're constructing types dynamically and want the explicit `typing.Union` object.

Step-by-step thought process: 1. "Both express 'value is X or Y' to the type checker. `Union[int, str]` is `typing.Union`; `int | str` produces `types.UnionType`. Mypy and Pyright treat them identically." 2. "In a 3.10+ codebase, `int | str` everywhere — no `typing` import for unions, reads cleanly, chains naturally — `int | str | bytes`." 3. "The new syntax also works in `isinstance` since 3.10. `isinstance(x, int | str)` is valid and avoids the older `isinstance(x, (int, str))` tuple form." 4. "Where the new syntax fails — Python < 3.10 evaluates `int | str` at runtime as an arithmetic operation on the types themselves, which raises `TypeError`. So on older Pythons, use `Union` or `from __future__ import annotations` to make annotations strings." 5. "For type aliases with the new syntax, use `MyType = int | str` directly. Or in 3.12+, `type MyType = int | str` — the new `type` statement makes aliases first-class and lazy-evaluated."

Edge cases to mention: - `Union[X]` with a single type is just `X`. Unions deduplicate, so `Union[int, int]` is `int` and `int | int` is `int`. Comes up in generated code. - Pattern matching on union types is a common 2026 pattern — `match value: case int(): ... case str(): ...`. Works seamlessly with `int | str` annotations. - For runtime type checks against complex unions involving generics, `isinstance` doesn't work — generics aren't reified. Use `typing.get_args` plus a manual check.

What NOT to say: - "Use `Union` for readability." Subjective and the wrong answer for 2026 — the pipe syntax is the modern norm. - "`isinstance(x, int | str)` doesn't work." Wrong in 3.10+ — it does work, no need for the tuple form anymore.

Common follow-up: "How would you write a type alias for 'an int, str, or a list of either'?" — `Item = int | str | list[int | str]`. Or in 3.12+ use the `type` statement.

```
# Modern union – 3.10+
def parse(value: int | str | bytes) -> str:
    if isinstance(value, int | str): # tuple-form not needed
        return str(value)
    return value.decode("utf-8")

# Type alias – 3.12+ syntax
type JsonScalar = int | float | str | bool | None

# Old form – still works, needed pre-3.10
from typing import Union
def parse_legacy(value: Union[int, str, bytes]) -> str: ...
```

Q38 · Protocol — Structural Subtyping

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Goldman · Topic: Typing

The question (interviewer's verbatim phrasing):

What's a `Protocol` in Python typing? How is it different from an ABC?

The 30-second answer: A `Protocol` (PEP 544, 3.8+) is "duck typing with type hints" — a class that defines a shape (methods, attributes) and any class with that shape satisfies it, without needing to inherit from it. ABCs require explicit inheritance or registration; Protocols match by structure alone. So Protocol is how you say "anything with a `.read()` method that returns bytes" without forcing the implementer to import your module.

Step-by-step thought process: 1. "With ABCs the relationship is nominal — you must subclass or call `MyABC.register(...)` to be considered a subclass. With Protocols the relationship is structural — if the methods match, you're a subclass for type-check purposes." 2. "The win is loose coupling. I define `class SupportsRead(Protocol): def read(self) -> bytes: ...`. Now any file object, network socket, or in-memory buffer with a matching `read` is accepted, even if the author of those classes never heard of my protocol." 3. "Protocols are checked at static-analysis time by mypy or pyright — at runtime they don't do anything by default. So `isinstance(x, MyProtocol)` raises `TypeError` unless you decorate the protocol with `@runtime_checkable`." 4. "`@runtime_checkable` enables `isinstance` checks but only validates that the methods exist by name — it doesn't check signatures, return types, or attribute types. So runtime check is shallow; trust the static checker for actual correctness." 5. "Practical use case — third-party objects you can't modify. I receive a `cache` argument that I want to assume has `.get()` and `.set()`. Define `class CacheProtocol(Protocol): def get(self, key: str) -> Any: ...; def set(self, key: str, value: Any) -> None: ...`. Any cache-like object now type-checks, including Redis clients, dict wrappers, and test doubles."

Edge cases to mention: - A class with extra methods still satisfies a Protocol — structural subtyping is one-way. The class needs to have at least the protocol's methods. - For attributes (not methods), declare them on the Protocol with type annotations: `class Named(Protocol): name: str`. The static checker verifies attribute access. - Generic Protocols are supported — `class Container(Protocol[T]): def get(self) -> T: ...`. Useful for typed containers.

What NOT to say: - "Protocol and ABC are the same thing." Misses the explicit-vs-structural distinction that's the whole point. - "`isinstance` always works on a Protocol." Only works on `@runtime_checkable` ones, and even then it's name-only.

Common follow-up: "When would you still prefer an ABC?" — When you want to share concrete implementation alongside the interface, or when you want to enforce nominal inheritance for clarity (e.g., a framework requires you to subclass `BaseView`).

```

from typing import Protocol, runtime_checkable

@runtime_checkable
class CacheProtocol(Protocol):
    def get(self, key: str) -> bytes | None: ...
    def set(self, key: str, value: bytes, ttl: int = 60) -> None: ...

def fetch_with_cache(cache: CacheProtocol, url: str) -> bytes:
    cached = cache.get(url)
    if cached is not None:
        return cached
    body = http.get(url).content
    cache.set(url, body, ttl=300)
    return body

# These all satisfy CacheProtocol without inheriting from it:
# - redis.Redis (has get + set with matching shape)
# - a dict-based test double
# - django.core.cache.cache

```

Q39 · if name == "main":

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture · Topic: Modules

The question (interviewer's verbatim phrasing):

What does `if __name__ == "__main__":` actually do? Why is it there?

The 30-second answer: Every module gets a `__name__` attribute set automatically. When you run a file directly with `python myfile.py`, `__name__` is the string `"__main__"`. When you import it from another file, `__name__` is the module's dotted name like `"myproject.utils"`. The `if __name__ == "__main__":` block lets a file behave like a library when imported and like a script when run directly — only the script-mode code runs at the bottom.

Step-by-step thought process: 1. "Python sets `__name__` per-module at import or load time. For the file you invoked from the command line, the name is `__main__` — anything else gets its real name." 2. "Without the guard, code at module top level runs every time the file is imported. So if `payments.py` had `print('starting up')` at the bottom and any other file did `import payments`, that line would fire on import. Almost always wrong." 3. "The pattern lets a module double as a library and a CLI entry point. `from payments import charge_card` imports the function. `python payments.py --order 12345` runs the script block — calling `main()` or whatever — without polluting library users." 4. "It also matters for `multiprocessing` on Windows and macOS — the `spawn` method re-imports your script in each worker. Without the guard, the workers run your main code recursively. Forgetting the guard is the textbook 'multiprocessing fork bomb' on Windows." 5. "Best practice — keep

`if __name__ == '__main__':` minimal. Define a `main()` function with the real logic, then the guard is just `if __name__ == '__main__': main()`. Easier to test, easier to import the entry point."

Edge cases to mention: - For packages, `python -m mypackage` runs

`mypackage/__main__.py` — same convention, different file name. - `__name__` is also

`"__main__"` when running in the REPL or via `python -c`, so the guard fires there too. -

`unittest.main()` and `argparse` examples are the most common things people put behind the guard.

What NOT to say: - "It's just a Python convention with no real purpose." Wrong — without it your library code runs side effects on import. - "It's only needed for scripts." Half-right; multiprocessing makes it required even for library-shaped modules that spawn workers.

Common follow-up: "What's the equivalent for a package?" — Create a `__main__.py` inside the package; `python -m mypackage` runs it. Used by tools like `pip`, `http.server`, `unittest`.

```
# payments.py – library + CLI in one file
def charge_card(order_id: int, amount: int) -> bool:
    """Importable from elsewhere."""
    ...

def main() -> int:
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--order", type=int, required=True)
    parser.add_argument("--amount", type=int, required=True)
    args = parser.parse_args()
    ok = charge_card(args.order, args.amount)
    return 0 if ok else 1

if __name__ == "__main__":
    raise SystemExit(main())
```

Q40 · Absolute vs Relative Imports

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Freshworks, Cred · Topic: Modules

The question (interviewer's verbatim phrasing):

When do you use absolute imports versus relative imports? How do you deal with a circular import?

The 30-second answer: Absolute imports — `from myproject.payments import charge_card` — name the full module path. Relative imports — `from . import sibling` or `from .submodule import thing` — name the location relative to the current package. Use absolute for clarity in application code; relative imports are useful inside a package to make it relocatable. Circular imports usually mean two modules import each other at top level — fix by deferring one of them inside a function, restructuring the shared dependency into a third module, or importing the module itself rather than a name from it.

Step-by-step thought process: 1. "PEP 328 makes absolute the default. `import os`, `from myapp.utils import foo` — these are absolute. Relative imports need the dot prefix — `from . import sibling`, `from ..core import db`." 2. "Relative imports only work inside a package — that is, a directory with `__init__.py` (or a namespace package). They use the current module's package as the starting point. So a single script file at the top level can't use relative imports." 3. "My default is absolute imports in application code — they survive moves of the caller, they read clearly in stack traces, they don't need the reader to know the current package. Relative imports earn their keep inside reusable packages where you don't want to hard-code the top-level name." 4. "Circular imports — A imports B, B imports A. Python doesn't crash, but one of them sees a half-finished module and gets an `ImportError` or `AttributeError` on the missing name. Three fixes, ranked by quality." 5. "Best — restructure. The shared thing probably belongs in a third module that both import. Second — defer the import. Move `from .b import thing` inside the function that uses it, so it runs only when called, by which time both modules are fully loaded. Third — import the module not the name. `from . import b` then `b.thing` — works because the module object exists even when half-loaded; the attribute lookup happens later."

Edge cases to mention: - `from . import x` works during module load when `x` is a submodule but fails when `x` is just a name defined later in `__init__.py`. Order of imports in `__init__.py` matters. - Type-only circular imports — types you need only for annotations — can be deferred with `if TYPE_CHECKING: from typing`. Adds zero runtime cost. - Namespace packages (no `__init__.py`) introduced in PEP 420 are now standard; relative imports work the same way as long as the directory is on `sys.path`.

What NOT to say: - "Use relative imports always — they're more Pythonic." Subjective and increasingly out of step with how large codebases organize themselves. - "Circular imports mean your design is wrong." Sometimes true but a lazy answer — sometimes two modules genuinely have a bidirectional relationship and a deferred import is the right tool.

Common follow-up: "Show me the `TYPE_CHECKING` trick for type-only circulars." — Wrap the import in `if TYPE_CHECKING:` and use a string annotation or `from __future__ import annotations`.

```
# myapp/payments.py – circular with myapp/orders.py
from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    # Only imported for type-checking, not at runtime
    from .orders import Order

def charge(order: Order) -> bool:
    # Deferred runtime import – both modules fully loaded by call time
    from .orders import mark_order_paid
    if _gateway.charge(order.amount):
        mark_order_paid(order)
        return True
    return False
```

Section 5 — Concurrency basics

The ten questions every Python interview pulls from when concurrency comes up. The GIL question alone shows up in roughly four out of five backend rounds at Razorpay, Swiggy, Cred, Zerodha and the GCC panels (JP Morgan, Goldman, Walmart Labs). Data-role buyers see fewer of these, but the top three — GIL, threading-vs-multiprocessing-vs-asyncio, and `async` / `await` — show up there too. Read it the way you'd skim a friend's notes the night before — fast first, then slow on the bits you can't say out loud yet.

These are the actual phrasings interviewers use. The 30-second answer is what you say first when the question lands; the step-by-step is the layered version for when they say "go deeper."

Q41 · The GIL in 60 seconds

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Cred · Topic: GIL

The question (interviewer's verbatim phrasing):

Explain the GIL to me in 60 seconds. What is it, why does it exist, and when does it actually hurt you?

The 30-second answer: The GIL is a single mutex inside CPython that only one thread can hold at a time when running Python bytecode. It exists because CPython's memory management — reference counting on every object — isn't thread-safe, and a single global lock was the cheapest way to keep it correct. It hurts you on CPU-bound multi-threading: four threads doing pure Python math run at roughly

the same speed as one. For I/O-bound work it doesn't hurt, because threads release the GIL while waiting on the network or disk.

Step-by-step thought process: 1. "The GIL — Global Interpreter Lock — is a CPython implementation detail, not a Python language feature. Jython and IronPython have no GIL. PyPy has one too, similar to CPython's." 2. "Mechanically, it's a mutex around the bytecode interpreter. A thread must hold the GIL to execute Python bytecode. Threads switch every few milliseconds, and they also release the GIL on I/O calls like socket reads or file reads." 3. "Why does it exist — refcounting. Every Python object has a reference count that gets touched on every assignment, every function call, every del. Without a single lock you'd need atomic operations or per-object locks, both of which would slow down single-threaded code, which is the common case." 4. "Where it bites — CPU-bound multi-threading. A NumPy matrix multiply releases the GIL inside the C extension, so threading helps. A pure-Python loop crunching numbers doesn't release the GIL, so threading buys you nothing — actually slightly less than nothing due to lock contention." 5. "The escape hatches today — `multiprocessing` for CPU work, `asyncio` for I/O concurrency without threads, C extensions that release the GIL, and from 3.13 the experimental free-threaded build (Q50)."

Edge cases to mention: - The GIL is released around C-extension calls that opt in via `Py_BEGIN_ALLOW_THREADS` — NumPy, lxml, hashlib, requests' underlying socket calls all do this. - The check interval changed from instruction-count (every 100 bytecodes, old) to time-based (every 5ms by default, since 3.2). `sys.setswitchinterval()` controls it. - "GIL released on I/O" is the actual mechanism behind why threading still works for I/O-bound code — the waiting thread parks, another runs.

What NOT to say: - "The GIL means Python can't do concurrency." — Wrong. It means Python can't do CPU-parallel threading. Concurrency via async, multiprocessing, and threaded I/O is unaffected. - "Python is slow because of the GIL." — Mixes two things. CPython is slow because of interpretation overhead and dynamic typing; the GIL is a separate issue that only matters for CPU-bound threading.

Common follow-up: "If the GIL only allows one thread to run Python at a time, why is `threading` ever useful?" — Because most real workloads are I/O-bound: hitting a DB, calling an API, reading a file. Threads release the GIL while waiting on those, so another thread runs. The GIL is only a wall for pure-Python CPU work.

```
# Demonstrating where the GIL bites (CPU-bound) vs doesn't (I/O-bound)
import threading, time

# CPU-bound: threads do not speed this up under the GIL
def crunch(n):
    s = 0
    for i in range(n):
        s += i * i
    return s

# I/O-bound: threads DO help because each releases the GIL while waiting
def fetch(url):
    import urllib.request
    return urllib.request.urlopen(url, timeout=5).read()
```

Q42 · Threading vs multiprocessing vs asyncio — the decision matrix

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Concurrency

The question (interviewer's verbatim phrasing):

I want to speed up my Python program. How do I pick between threading, multiprocessing, and asyncio?

The 30-second answer: Ask one question first — is the work CPU-bound or I/O-bound? CPU-bound goes to **multiprocessing** (one process per core, sidestep the GIL). I/O-bound goes to **asyncio** if you control the call sites and the libraries are async-aware, or **threading** if you're stuck with sync libraries or only have a handful of concurrent calls. Don't reach for threading for CPU work and don't reach for asyncio if your DB driver is blocking.

Step-by-step thought process: 1. "Step one is classifying the workload. CPU-bound — number crunching, parsing, image processing without C extensions, anything where the bottleneck is the processor. I/O-bound — DB queries, HTTP calls, file reads, message-queue waits. The mental model is: is your code busy or waiting?" 2. "CPU-bound → multiprocessing. Each process has its own Python interpreter and its own GIL, so a 4-core machine gets a near-4x speedup for embarrassingly parallel work. Cost: process startup is slow, and you need to pickle data across the process boundary." 3. "I/O-bound → asyncio first if the call sites support it. One event loop on one thread can handle thousands of concurrent connections because each task parks itself while waiting. Cost: needs async-native libraries — **httpx** not **requests**, **aiomysql** not **mysql.connector**, etc." 4. "I/O-bound but stuck with sync libraries → threading. The classic shape is **ThreadPoolExecutor** with a dozen workers calling a sync API. The GIL doesn't block you because each thread is waiting on I/O most of the time." 5. "The honest answer for a 2026 Python service — pick asyncio for the framework layer (FastAPI, aiohttp) and offload the CPU spikes to a process pool via **asyncio.to_thread** or a separate worker process."

Edge cases to mention: - "Mostly I/O, occasionally CPU" — wrap the CPU bit in `asyncio.to_thread()` so the event loop doesn't stall. - `multiprocessing` on Windows and on macOS (since 3.8) defaults to "spawn" not "fork" — it re-imports your module in each worker, which trips up code that runs at import time. - A thread pool with 100 workers on a sync HTTP library is still way slower than asyncio with 1000 concurrent tasks — threads cost about 1-8 MB of stack each.

What NOT to say: - "Always use asyncio, it's the modern way." — Wrong if your code is CPU-bound or your libraries are sync. - "Threading is dead because of the GIL." — Threading is the right tool for I/O-bound work with sync libraries; it's not dead at all.

Common follow-up: *"My pandas script processes a 5 GB CSV in chunks and it's pegging one core. Which model?"* — Multiprocessing. Use `multiprocessing.Pool` to fan chunks across cores, or PyArrow + Polars which already parallelise internally.

```
# The decision matrix in code
# CPU-bound -> multiprocessing
from multiprocessing import Pool
def heavy(n): return sum(i*i for i in range(n))

# I/O-bound (async-native libs) -> asyncio
import asyncio, httpx
async def fetch_all(urls):
    async with httpx.AsyncClient() as c:
        return await asyncio.gather(*(c.get(u) for u in urls))

# I/O-bound (sync libs) -> threading
from concurrent.futures import ThreadPoolExecutor
import requests

# IMPORTANT: on macOS/Windows the default start method is "spawn",
# which re-imports this module in every worker. Anything that
# instantiates Pool/Executor must live under the __main__ guard
# or you'll spawn workers recursively (fork bomb - see Q39).
if __name__ == "__main__":
    urls = ["https://example.com"] * 10
    with Pool(4) as p:
        results = p.map(heavy, [10_000_000] * 4)
    with ThreadPoolExecutor(max_workers=10) as ex:
        pages = list(ex.map(requests.get, urls))
```

Q43 · `async` and `await` basics

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, Freshworks, Zoho · Topic: asyncio

The question (interviewer's verbatim phrasing):

Explain what `async def` and `await` actually do. What's the difference between a coroutine and a regular function?

The 30-second answer: `async def` defines a coroutine function — calling it doesn't run the body, it returns a coroutine object you have to schedule on an event loop. `await` is what hands control back to the loop until the awaited thing — another coroutine, a Future, a Task — is ready. Between `await` points your code runs straight through, single-threaded; at each `await` the loop can switch to another task that's ready.

Step-by-step thought process: 1. "A regular function call runs the body and returns a value. An `async def` call returns a coroutine object that hasn't run yet. You either `await` it inside another coroutine or hand it to `asyncio.run()`." 2. "`await x` does two things — if `x` isn't done, it yields control back to the event loop, registering itself to resume when `x` is done. If `x` is already done, it returns the value immediately without yielding." 3. "Mental model — coroutines cooperate. They voluntarily release control at `await` points. There's no preemption, no thread context switch, no GIL juggling. One task runs at a time on one thread; the loop picks the next ready task each time the current one parks." 4. "That's why blocking calls inside a coroutine are a sin. `time.sleep(5)` inside async code stops the whole event loop for 5 seconds — every other task waits. Use `await asyncio.sleep(5)` instead." 5. "What you can `await` — anything that implements `__await__`. In practice: another coroutine, a `Task`, a `Future`, or anything from an async library that returns an awaitable."

Edge cases to mention: - Forgetting `await` is the most common bug — `coro()` without `await` returns a coroutine object that warns at GC time: "coroutine was never awaited." - You can `await` a coroutine only once. After it finishes, awaiting it again raises `RuntimeError`. Wrap in a `Task` if you want multiple awaiters. - `async def` functions can contain `return` and the return value is what `await` produces.

What NOT to say: - "`await` is like `yield`." — Related historically but say "yields control to the event loop" instead. Saying "it's like yield" usually leads the interviewer to ask follow-ups about generators that you don't need. - "asyncio uses threads underneath." — Wrong by default. The event loop is single-threaded. `asyncio.to_thread()` is the explicit escape hatch.

Common follow-up: "What happens if I put a `time.sleep(10)` inside a coroutine?" — You block the entire event loop for 10 seconds. Every other task — every other HTTP request your server is handling — waits. Use `await asyncio.sleep(10)` for async sleep, or `await asyncio.to_thread(time.sleep, 10)` if you really need to call a blocking thing.

```
import asyncio

async def fetch(name, seconds):
    print(f"{name} start")
    await asyncio.sleep(seconds) # yields to loop; other tasks run
    print(f"{name} done")
    return name

async def main():
    # gather schedules both; total time = max(1, 2), not 1 + 2
    results = await asyncio.gather(fetch("A", 1), fetch("B", 2))
    print(results)

asyncio.run(main())
```

Q44 · `asyncio.run()` — the entry point

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Freshworks, Walmart Labs · Topic: asyncio

The question (interviewer's verbatim phrasing):

What does `asyncio.run()` do? Why don't I manually create an event loop anymore?

The 30-second answer: `asyncio.run(coro)` is the standard entry point — it creates a fresh event loop, runs the coroutine until it completes, cancels any leftover tasks, closes the loop, and returns the result. Before 3.7 you wrote three lines of boilerplate to do this; now `asyncio.run` is the one-line replacement and the docs say to use it for top-level entry.

Step-by-step thought process: 1. "`asyncio.run(main())` is what you call from your `if __name__ == '__main__':` block. It's a synchronous function — you call it from sync code, and it blocks until the coroutine finishes." 2. "Internally it does what we used to write by hand: `loop = asyncio.new_event_loop()`, `asyncio.set_event_loop(loop)`, `loop.run_until_complete(coro)`, then cleanup. The cleanup matters — it cancels any pending tasks, runs their cancellation handlers, shuts down async generators, then closes the loop." 3. "The rule — call it exactly once, at the top of your program. Don't call it inside another coroutine, don't call it inside a loop, don't call it from a thread that already has a loop. It assumes it owns the loop lifecycle." 4. "Why not manual loops anymore — because forgetting the cleanup steps causes resource leaks, async generators that never shut down, warnings at exit. `asyncio.run` was added precisely to make the right thing the default thing."

Edge cases to mention: - Calling `asyncio.run()` from inside a running event loop raises `RuntimeError: asyncio.run() cannot be called from a running event loop`. Use `await main_coro()` instead — you're already in async-land. - Frameworks like FastAPI, aiohttp,

and Jupyter manage the loop for you — don't call `asyncio.run()` from a FastAPI handler. - The `debug=True` flag turns on debug mode for the loop — logs slow callbacks, catches coroutines that were never awaited. Worth turning on in development.

What NOT to say: - "I always create the loop manually so I have more control." — Sounds senior but usually means you'll forget to close it. The default is correct for 99% of programs. - "`asyncio.run` is just `loop.run_until_complete` ." — Misses the cleanup phase, which is the whole point.

Common follow-up: "What if I'm in a Jupyter notebook and `asyncio.run` says the loop is already running?" — Jupyter runs its own loop. Inside a notebook cell, just `await main_coro()` directly — top-level await works in Jupyter. Outside Jupyter, that won't compile.

```
import asyncio

async def main():
    await asyncio.sleep(0.1)
    return "done"

# The right way – one call at the top of the program
if __name__ == "__main__":
    result = asyncio.run(main())
    print(result)

# What asyncio.run is doing under the hood (don't write this anymore):
# loop = asyncio.new_event_loop()
# try:
#     result = loop.run_until_complete(main())
# finally:
#     loop.close()
```

Q45 · `asyncio.gather()` vs `asyncio.TaskGroup()` — structured concurrency

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs, Postman · Topic: asyncio

The question (interviewer's verbatim phrasing):

When would you use `asyncio.gather` versus `asyncio.TaskGroup` ? What's "structured concurrency"?

The 30-second answer: `gather` runs awaitables concurrently and returns their results in order; if one fails, the others keep running by default and you lose visibility on which failed. `TaskGroup` (3.11+) is structured concurrency — when you exit the `async with` block, every task is guaranteed done. If one

task raises, the rest get cancelled and you get an `ExceptionGroup` with all errors. For new code, prefer `TaskGroup`.

Step-by-step thought process: 1. "`gather` is the older API — `await asyncio.gather(coro1, coro2, coro3)`. Runs them concurrently, returns a list of results in the order you passed them. Exception handling is awkward: by default the first exception propagates but the other tasks keep running until they finish, and their results are dropped." 2. "`TaskGroup` arrived in 3.11 as the structured-concurrency answer. You open `async with asyncio.TaskGroup() as tg:`, call `tg.create_task(coro)` for each one, and the `__aexit__` waits for all of them. If any task fails, the others are cancelled and you get an `ExceptionGroup` containing every failure." 3. "Why structured concurrency matters — tasks can't outlive their scope. With raw `gather` you can accidentally fire-and-forget a task and have it running long after the function returns, which is how you get zombie connections and resource leaks. `TaskGroup` makes the scope explicit." 4. "The migration rule — if all your tasks are short and you want all results, swap `gather` for `TaskGroup` and read each task's `.result()` after the `async with` block. If you want truly fire-and-forget tasks, you're outside structured concurrency on purpose — keep a reference and handle cancellation yourself."

Edge cases to mention: - `gather(..., return_exceptions=True)` returns exceptions as values instead of raising — useful for "best-effort" calls, but you lose stack traces. - `TaskGroup` cancels siblings on any task exception, including `asyncio.CancelledError` — be careful with custom cancellation flows. - You can still nest a `TaskGroup` inside another one — exceptions bubble up correctly through nested groups as nested `ExceptionGroup`s.

What NOT to say: - "They're basically the same." — They have very different exception semantics and lifetime guarantees. - "`gather` is deprecated." — It isn't. It's still useful for simple cases and broadly compatible with older codebases.

Common follow-up: "How do you handle errors from a `TaskGroup`?" — Wrap the `async with` in `try/except*` (3.11+) for typed `ExceptionGroup` matching. `except* ValueError` catches only the ValueErrors; `except* (TimeoutError, ConnectionError)` catches both, leaving anything else to propagate.

```
import asyncio

async def fetch(name, delay, fail=False):
    await asyncio.sleep(delay)
    if fail:
        raise ValueError(f"{name} failed")
    return name

# Modern: TaskGroup with structured exception handling
async def main_taskgroup():
    try:
        async with asyncio.TaskGroup() as tg:
            t1 = tg.create_task(fetch("A", 0.1))
            t2 = tg.create_task(fetch("B", 0.2))
            t3 = tg.create_task(fetch("C", 0.3, fail=True))
        print(t1.result(), t2.result())
    except* ValueError as eg:
        for err in eg.exceptions:
            print(f"caught: {err}")

# Older: gather – keeps running siblings, less explicit
async def main_gather():
    results = await asyncio.gather(
        fetch("A", 0.1), fetch("B", 0.2), fetch("C", 0.3, fail=True),
        return_exceptions=True,
    )
    print(results) # ['A', 'B', ValueError('C failed')]

asyncio.run(main_taskgroup())
```

Q46 · `asyncio.timeout()` vs `asyncio.wait_for()`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, JP Morgan · Topic: asyncio

The question (interviewer's verbatim phrasing):

How do you put a timeout on an async operation? What changed in 3.11?

The 30-second answer: The modern way (3.11+) is `async with asyncio.timeout(5):` — a context manager that cancels everything inside the block if 5 seconds pass. The older way is `await asyncio.wait_for(coro, timeout=5)` — wraps one coroutine and raises `TimeoutError` if it doesn't finish. Use the context manager for new code: it's composable, you can reschedule it, and it cleanly wraps a whole block.

Step-by-step thought process: 1. "`asyncio.wait_for(coro, timeout)` is the pre-3.11 API. You pass it one awaitable and a timeout in seconds. If the coroutine doesn't finish in time, it cancels the

coroutine and raises `TimeoutError`. Simple but limited — works on exactly one awaitable." 2. "`asyncio.timeout(seconds)` is the 3.11+ context manager. You write `async with asyncio.timeout(5): ... do stuff ...`. Anything inside, including multiple awaits and nested calls, is bound by the same deadline. Hitting the deadline raises `TimeoutError` from the `async with` line." 3. "Why the new API is better — composability. You can wrap a section of code that calls many things. You can also reschedule the deadline with `asyncio.timeout(None).reschedule(when)` if a long-running operation needs more time. `wait_for` can't do either." 4. "Migration rule — for new code, use `asyncio.timeout()`. Keep `wait_for` only if you're stuck on 3.10 or earlier."

Edge cases to mention: - Both raise the built-in `TimeoutError` (in 3.11+, `asyncio.TimeoutError` is an alias for the built-in). Catch `TimeoutError`, not `asyncio.TimeoutError` specifically. - If the inner code catches `CancelledError` and swallows it, neither timeout API can interrupt it — bad code defeats good APIs. - The context manager `asyncio.timeout_at(deadline)` lets you specify an absolute deadline (loop time) instead of a relative duration — useful when propagating a deadline through layers.

What NOT to say: - "I'll just use `signal.alarm()`." — Wrong for async. `signal.alarm` is process-wide, doesn't compose with the event loop, and doesn't work on Windows. - "`wait_for` and `timeout` are interchangeable." — They aren't. One wraps a single awaitable; the other wraps an arbitrary block. They also have subtle differences in cancellation behaviour when you stack multiple timeouts.

Common follow-up: "What if I want a timeout for the whole batch but no timeout for any individual call?" — That's exactly what `async with asyncio.timeout(N):` plus a `TaskGroup` inside gives you. The batch deadline applies; individual tasks aren't separately constrained.

```
import asyncio

async def slow_call():
    await asyncio.sleep(10)
    return "done"

# Modern: context manager – composable
async def with_timeout_block():
    try:
        async with asyncio.timeout(2):
            result = await slow_call()
            # could await multiple things here, all bounded by the same 2s
            return result
    except TimeoutError:
        return "timed out"

# Older: wait_for – wraps one coroutine
async def with_wait_for():
    try:
        return await asyncio.wait_for(slow_call(), timeout=2)
    except TimeoutError:
        return "timed out"

asyncio.run(with_timeout_block())
```

Q47 · `threading.Lock` — why the GIL doesn't replace it

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan, Walmart Labs · Topic: Threading

The question (interviewer's verbatim phrasing):

If the GIL means only one thread runs at a time, why do I still need `threading.Lock` ?

The 30-second answer: The GIL guarantees one bytecode instruction is atomic — not one Python statement. `counter += 1` looks atomic but compiles to four bytecodes: load, push 1, add, store. A thread switch between them, and you lose increments. `threading.Lock` makes a compound operation atomic at the lock-acquire boundary, which the GIL alone can't.

Step-by-step thought process: 1. "The GIL's atomicity guarantee is per-bytecode, not per-statement. `dis.dis('counter += 1')` shows you four operations. Between any two of those, the GIL can switch to another thread." 2. "Concrete failure — two threads each running `counter += 1` a million times. The expected end state is 2,000,000. The actual is anywhere from ~1.0M to ~1.9M, because each thread reads-then-writes and they overwrite each other's increments." 3. " `threading.Lock` fixes this by serialising the compound operation. Wrap `counter += 1` in `with lock:` and only one thread

holds the lock at a time — the rest queue." 4. "What's safe without a lock — single-bytecode operations: `list.append(x)`, `dict[k] = v`, `set.add(x)`. These compile to one `CALL_FUNCTION` or one `STORE_SUBSCR`. The GIL guarantees they finish before a switch. Multi-step operations like `dict[k] = dict[k] + 1` or `list[-1] = ...` are not safe." 5. "Practitioner rule — if your operation involves a read followed by a write to the same shared state, you need a lock. If it's a single mutating call on a thread-safe container, you're usually fine, but write a test."

Edge cases to mention: - `queue.Queue` is the goto thread-safe container — internally locked, designed for producer/consumer patterns. Reach for it before reaching for raw `Lock`. - `threading.RLock` is reentrant — same thread can acquire it multiple times. Useful when a locked method calls another locked method on the same object. - `with lock:` is the idiomatic acquire/release — always prefer it over `lock.acquire()` / `lock.release()` because it handles exceptions.

What NOT to say: - "The GIL makes Python thread-safe." — Wrong. It makes bytecode execution atomic, which is a much weaker guarantee. - "I'll just use atomic operations." — Python has no atomic-int type in the stdlib (Java's `AtomicInteger` has no direct equivalent). You need `Lock` or `queue.Queue`.

Common follow-up: "Show me with `dis` why `counter += 1` isn't atomic." — Run `import dis; dis.dis('counter += 1')` and you see `LOAD_NAME`, `LOAD_CONST 1`, `BINARY_OP`, `STORE_NAME` — four steps, three switch points.

```
import threading

counter = 0
lock = threading.Lock()

def increment_unsafe():
    global counter
    for _ in range(100_000):
        counter += 1 # LOAD/ADD/STORE – not atomic

def increment_safe():
    global counter
    for _ in range(100_000):
        with lock:
            counter += 1 # serialised

threads = [threading.Thread(target=increment_safe) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(counter) # 400_000 with lock; usually less without
```

Q48 · multiprocessing.Pool — when to reach for it

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Swiggy, Walmart Labs, Target · Topic: Multiprocessing

The question (interviewer's verbatim phrasing):

When do you reach for `multiprocessing.Pool` ? What are the gotchas?

The 30-second answer: You reach for `multiprocessing.Pool` when the work is CPU-bound and embarrassingly parallel — a `map`-shaped problem over a list. Each worker is a separate Python process with its own GIL, so you actually use all cores. The gotchas — function and arguments must be picklable, IPC has real overhead (don't fan tiny tasks out), and on macOS/Windows the worker re-imports your module so import-time code runs N times.

Step-by-step thought process: 1. "Mental model — Pool spins up N worker processes (default = `os.cpu_count()`). You hand it an iterable, and it slices the work into chunks, pickles each chunk to a worker over a pipe, the worker runs your function, pickles the result back, and the main process collects." 2. "Right shape for Pool — many independent items, each item involves enough CPU work to dwarf the pickle-and-pipe cost. Examples: parsing 10k files, running ML inference on a batch, transforming chunks of a large DataFrame." 3. "Wrong shape — short tasks. If each task is 0.5 ms of work plus 5 ms of IPC overhead, Pool slows you down. The threshold is roughly: each task should be at least 50-100 ms of CPU work for Pool to win." 4. "Pickling restrictions — the function and its arguments must be picklable. Lambdas, nested functions, and instance methods on un-importable classes all fail. Move the worker function to module scope. Same for arguments: NumPy arrays and DataFrames pickle fine; open file handles and DB connections don't." 5. "Start method matters — on Linux the default is `fork` (cheap, inherits memory). On macOS (since 3.8) and Windows the default is `spawn` (re-imports the module). Code under `if __name__ == '__main__':` is your friend; module-level side effects are your enemy."

Edge cases to mention: - `Pool.imap_unordered` returns results as they finish, which is faster when tasks have variable runtime — but you lose input order. - `chunksize=` matters. Default 1 sends every item one-by-one — slow. Set `chunksize=100` or higher for short tasks to amortise IPC cost. - For shared read-only large data (a model, a lookup table), prefer `multiprocessing.shared_memory` or load it once per worker via an initializer rather than re-pickling per call.

What NOT to say: - "Just use Pool for everything." — IPC overhead and pickle costs make it slower than threading or asyncio for I/O work. - "Pool gives you true parallelism." — True for CPU; misleading for I/O. Multiprocessing still uses real OS threads/processes, but for I/O-bound work asyncio is cheaper.

Common follow-up: "How do you share state across workers?" — Three options.

`multiprocessing.Manager()` for shared dicts/lists (slow, proxied via IPC).

`multiprocessing.shared_memory` for raw bytes/NumPy arrays (fast, since 3.8). Or just return results from each worker and combine in the main process — usually the cleanest answer.

```

from multiprocessing import Pool, cpu_count

# Module-level function — picklable
def transform(record):
    # CPU-bound work: parsing, hashing, ML inference
    return record["id"], hash(record["payload"])

if __name__ == "__main__":
    records = [{"id": i, "payload": "data" * 1000} for i in range(10_000)]
    with Pool(cpu_count()) as p:
        results = p.map(transform, records, chunksize=100)
    print(len(results))

```

Q49 · `concurrent.futures` — the executor pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, JP Morgan, Walmart Labs · Topic: Concurrency

The question (interviewer's verbatim phrasing):

Walk me through `concurrent.futures`. What's the difference between `ThreadPoolExecutor` and `ProcessPoolExecutor`?

The 30-second answer: `concurrent.futures` is the unified high-level API for parallel work. Both executors share the same interface: `submit(fn, *args)` returns a `Future`, `map(fn, iterable)` returns results in order, `as_completed(futures)` yields them as they finish. `ThreadPoolExecutor` runs in threads (good for I/O-bound with sync libraries); `ProcessPoolExecutor` runs in processes (good for CPU-bound). Same code shape, different concurrency model.

Step-by-step thought process: 1. "The big win of `concurrent.futures` is the unified shape. You write the code with one executor, and switching to the other is a one-line change. Useful when you're prototyping and want to measure threads vs processes for your specific workload." 2. "`ThreadPoolExecutor` — N threads, default `min(32, os.cpu_count() + 4)`. Threads share memory, are cheap to start, and are the right answer for I/O-bound work where your libraries are sync. Each thread releases the GIL during I/O." 3. "`ProcessPoolExecutor` — N processes, default `os.cpu_count()`. Each process has its own interpreter. Use it for CPU-bound work. Comes with all the multiprocessing gotchas: picklability, spawn overhead, import-time code re-runs." 4. "Three ways to read results. `executor.map(fn, iterable)` for in-order results. `[ex.submit(fn, x) for x in items]` then `as_completed(futures)` for results as they finish. `Future.result(timeout=)` for one specific result with optional timeout." 5. "Practitioner rule — for

ad-hoc parallel work, reach for `concurrent.futures` first. For long-running task systems and async-native services, drop down to `threading` / `multiprocessing` / `asyncio` directly."

Edge cases to mention: - `Future.result()` re-raises any exception the worker threw — exceptions don't disappear silently. - `executor.map()` raises on the first failed item but only when you iterate past it — wrap the iteration in try/except. - Always use `with ThreadPoolExecutor() as ex:` — the context manager shuts down the pool and joins all workers on exit. Leaking executors leaks threads.

What NOT to say: - "`ThreadPoolExecutor` is faster than raw threading." — It's the same threads; just a nicer API. Throughput is identical. - "Just pick the executor with more workers." — Picking more workers than your CPU/IO can handle thrashes. `cpu_count()` for `ProcessPool`; experimentally tuned for `ThreadPool` depending on I/O concurrency.

Common follow-up: "In a FastAPI handler, can I use `ThreadPoolExecutor` to call a blocking library?" — Yes, but the FastAPI idiomatic way is `await asyncio.to_thread(blocking_fn, *args)`, which uses the default loop executor under the hood. Same idea, less ceremony.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor, as_completed
import requests

# I/O-bound — threads
def fetch(url):
    return requests.get(url, timeout=5).status_code

urls = ["https://example.com"] * 50
with ThreadPoolExecutor(max_workers=10) as ex:
    for fut in as_completed([ex.submit(fetch, u) for u in urls]):
        print(fut.result())

# CPU-bound — processes (only the executor class changes)
def crunch(n):
    return sum(i * i for i in range(n))

with ProcessPoolExecutor() as ex:
    results = list(ex.map(crunch, [10_000_000] * 4))
```

Q50 · Python 3.13 free-threaded build (PEP 703)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Walmart Labs, Postman · Topic: GIL, 3.13

The question (interviewer's verbatim phrasing):

Have you heard about the no-GIL Python? What is it, what changes, and what would you say if I asked you about it tomorrow?

The 30-second answer: Python 3.13 ships an experimental "free-threaded" build — also called PEP 703 — that removes the GIL. CPU-bound multi-threading actually scales across cores in this build. It's opt-in: the standard CPython 3.13 still has the GIL; you have to install the free-threaded variant (`python3.13t`). For interviews today, the right thing to say is: it exists, it's experimental, single-threaded code is roughly 10-15% slower in the free-threaded build, and most C extensions need updates to be safe — so production-grade adoption is still 2-3 years out.

Step-by-step thought process: 1. "PEP 703 was accepted in 2023. It proposed removing the GIL while keeping CPython single-threaded performance acceptable. Python 3.13 (released October 2024) ships it as the `python3.13t` build — t for 'threaded'. Standard `python3.13` still has the GIL by default." 2. "What changes — CPU-bound threading actually parallelises. Four threads doing pure Python number crunching can use four cores. That's a genuinely new capability after 30 years." 3. "What's the cost — reference counting becomes biased (fast path) + atomic (slow path) instead of a single counter. Single-threaded code in the free-threaded build runs 5-15% slower than standard CPython 3.13 on most benchmarks. The Python team is closing the gap version by version." 4. "C extensions are the practical blocker. Any C extension that wasn't written for free-threading can have race conditions on internal state. NumPy, lxml, Pillow are working toward `Py_GIL_DISABLED`-aware builds, but coverage is patchy and the ecosystem will take a couple of releases to catch up." 5. "Interview answer today — 'yes, I know PEP 703 lands in 3.13 as the experimental free-threaded build. It removes the GIL for CPU parallelism. It's experimental, slower for single-threaded code, and most C extensions need rebuilds. I'd evaluate it for a CPU-bound service in 2027 but I'm not putting it in production yet.'"

Edge cases to mention: - The free-threaded build is enabled via a separate compile (`--disable-gil`) — Linux distros and Homebrew now ship both variants. `sys._is_gil_enabled()` returns `False` in the free-threaded build. - `multiprocessing` doesn't go away — it's still useful for fault isolation and bypassing memory contention even when threads scale. - 3.14 (October 2025) made the free-threaded build officially supported (no longer "experimental"); 3.15 (planned 2026) is expected to close more of the single-threaded perf gap.

What NOT to say: - "The GIL is dead." — It's not. The standard CPython build still has it, and will for years. Free-threading is opt-in and not the default. - "I'll just switch to free-threaded Python and remove all my multiprocessing code." — Don't. Your C extensions probably aren't ready, and single-threaded paths will get slower.

Common follow-up: "So should we use free-threaded Python for a new backend service?" — Probably not for a service launching in 2026 unless every C extension you depend on is free-threading-ready and you've benchmarked your single-threaded paths. For a pure-Python CPU-bound batch job with no critical extensions, it's worth evaluating. For a typical FastAPI service that's I/O-bound, asyncio is the right answer regardless of GIL changes.

```
# Detecting which Python build you're running
import sys

if sys.version_info >= (3, 13):
    # Available in 3.13+; True for standard build, False for free-threaded
    gil_on = sys._is_gil_enabled()
    print(f"GIL enabled: {gil_on}")
    print(f"Build: {'standard' if gil_on else 'free-threaded'}")

# Threading code reads identically – only behaviour changes
import threading

def crunch(n):
    return sum(i * i for i in range(n))

threads = [threading.Thread(target=crunch, args=(10_000_000,)) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
# Standard build: ~serial speed (GIL serialises)
# Free-threaded build: ~4x speedup on 4 cores
```